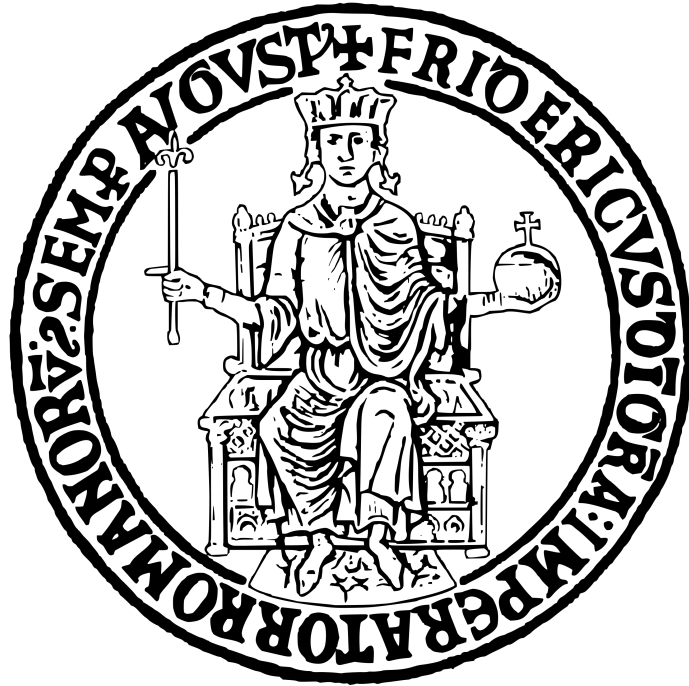




UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

Corso di Laurea Triennale in Ingegneria Informatica

INGEGNERIA DEL SOFTWARE

HAIRCARE STORE

Prof. Stefano Russo

a.a. 2025-2026

Studente:

Andrea Visconti N46007557

Indice

1	Specifiche informali	2
2	Analisi e specifica dei requisiti	2
2.1	Analisi nomi-verbi	2
2.2	Revisione dei requisiti	3
2.3	Glossario dei termini	4
2.4	Classificazione dei requisiti	5
2.4.1	Requisiti funzionali	5
2.4.2	Requisiti sui dati	5
2.4.3	Vincoli / Altri requisiti	6
2.5	Modellazione dei casi d'uso	6
2.5.1	Attori e casi d'uso	6
2.5.2	Diagramma dei casi d'uso	8
2.5.3	Scenari	8
2.6	Modellazione dei dati	10
2.6.1	Progettazione concettuale	10
2.7	Diagramma delle classi	10
2.8	Diagrammi di sequenza	12
2.9	Verifica della completezza dei requisiti	13
3	Stima dei costi	14
4	Piano di test funzionale	16
4.1	Test Suite	17
5	Progettazione	23
5.1	Progettazione della base di dati	23
5.2	Diagramma delle classi	25
5.3	Diagrammi di sequenza	26
6	Implementazione	27
6.1	Organizzazione dei package - Backend	27
6.2	Configurazione Backend	28
6.2.1	pom.xml	28
6.2.2	application.properties	28
6.3	Organizzazione dei package - Frontend	29
6.4	Configurazione Frontend	29
6.5	Database	30
6.6	Documentazione Javadoc	30
6.7	Metriche di dimensionamento del codice	41
7	Testing	42
7.1	Test strutturale	42
7.1.1	Complessità ciclomatica	42
7.1.2	Test di unità	45
7.2	Test Funzionale	46

1 Specifiche informali

Si vuole realizzare un'applicazione per la gestione del negozio online "HairCareStore" per la vendita di prodotti per la cura dei capelli (shampoo, balsami, maschere, oli, tinte, prodotti per lo styling e simili), con consegna a domicilio. Il sistema deve consentire a un impiegato di aggiungere, modificare o rimuovere i prodotti. Ogni prodotto è caratterizzato da un codice (es: "KER-SHAM250-DRY"), un nome (es: "Shampoo Nutriente Capelli Secchi 250ml"), una descrizione (es: "Shampoo idratante per capelli secchi e danneggiati, arricchito con cheratina"), una categoria (es: SHAMPOO, BALSAMO, MASCHERA, OLIO, TINTA, STYLING), un prezzo. L'impiegato inserisce inoltre i fattorini, registrando per ciascuno di essi nome, cognome, e-mail, numero di telefono. Per effettuare acquisti online, i clienti devono preventivamente registrarsi fornendo nome utente, password, indirizzo di spedizione, numero di telefono mobile e carta di credito. Il cliente visualizza il catalogo dei prodotti e può procedere direttamente all'acquisto. Il cliente effettua l'ordine procedendo al pagamento tramite un sistema di pagamento esterno e richiedendo la consegna a domicilio. Al completamento del pagamento, il sistema:

1. invia un messaggio al cellulare del cliente con il riepilogo dell'ordine effettuato e il prezzo pagato;
2. invia una notifica all'impiegato in negozio, che deve preparare il pacco per la consegna;
3. invia una notifica al fattorino, che deve ritirare il pacco dall'impiegato ed effettuare la consegna a domicilio.

L'ordine è caratterizzato da un ID, una data, il costo totale, uno stato (IN_CORSO, CONFERMATO, SPEDITO, CONSEGNATO) e il prodotto acquistato. Appena effettuata la consegna, il corriere segnala al sistema l'avvenuta consegna, e il sistema registra data e ora dell'effettiva consegna. Per finalità di marketing il direttore del negozio può richiedere al sistema di generare un report di tutti clienti che hanno effettuato acquisti nel mese precedente, con l'importo complessivo speso da ciascun cliente. Per l'approvvigionamento dei prodotti dai fornitori il direttore richiede mensilmente un report dei prodotti venduti con la relativa quantità totale. Al fine poi di valutare l'esigenza di assumere altri fattorini, il direttore può inoltre richiedere al sistema un report con i tempi medi di consegna per fattorino per mese (il tempo di una consegna è il tempo tra la notifica al fattorino e la effettiva consegna).

2 Analisi e specifica dei requisiti

2.1 Analisi nomi-verbi

Si vuole realizzare un'applicazione per la gestione del negozio online "HairCareStore" per la vendita di prodotti per la cura dei capelli (shampoo, balsami, maschere, oli, tinte, prodotti per lo styling e simili), con consegna a domicilio. Il sistema deve consentire a un impiegato di aggiungere, modificare o rimuovere i prodotti. Ogni prodotto è caratterizzato da un codice (es: "KER-SHAM250-DRY"), un nome (es: "Shampoo Nutriente Capelli Secchi 250ml"), una descrizione (es: "Shampoo idratante per capelli secchi e danneggiati, arricchito con cheratina"), una categoria (es: SHAMPOO, BALSAMO, MASCHERA, OLIO, TINTA, STYLING), un prezzo. L'impiegato inserisce inoltre i fattorini, registrando per ciascuno di essi nome, cognome, e-mail, numero di telefono. Per effettuare acquisti online, i clienti devono preventivamente registrarsi fornendo nome utente, password, indirizzo di spedizione, numero di telefono mobile e carta di credito. Il cliente visualizza il catalogo dei prodotti e può procedere direttamente all'acquisto. Il cliente effettua l'ordine procedendo al pagamento tramite un sistema di pagamento esterno e richiedendo la consegna a domicilio. Al completamento del pagamento, il sistema:

1. invia un messaggio al cellulare del cliente con il riepilogo dell'ordine effettuato e il prezzo pagato;
2. invia una notifica all'impiegato in negozio, che deve preparare il pacco per la consegna;
3. invia una notifica al fattorino, che deve ritirare il pacco dall'impiegato ed effettuare la consegna a domicilio.

L'ordine è caratterizzato da un ID, una data, il costo totale, uno stato (IN CORSO, CONFERMATO, SPEDITO, CONSEGNATO) e il prodotto acquistato. Appena effettuata la consegna, il corriere segnala al sistema l'avvenuta consegna, e il sistema registra data e ora dell'effettiva consegna. Per finalità di marketing il direttore del negozio può richiedere al sistema di generare un report di tutti clienti che hanno effettuato acquisti nel mese precedente, con l'importo complessivo speso da ciascun cliente. Per l'approvvigionamento dei prodotti dai fornitori il direttore richiede mensilmente un report dei prodotti venduti con la relativa quantità totale. Al fine poi di valutare l'esigenza di assumere altri fattorini, il direttore può inoltre richiedere al sistema un report con i tempi medi di consegna per fattorino per mese (il tempo di una consegna è il tempo tra la notifica al fattorino e la effettiva consegna).

Leggenda:

- Classe
- Attributo
- Attore
- Classe-Attore
- Funzionalità

2.2 Revisione dei requisiti

1. Il sistema deve consentire ad un impiegato di aggiungere nuovi prodotti nel catalogo.
2. Il sistema deve consentire ad un impiegato di modificare i prodotti esistenti nel catalogo.
3. Il sistema deve consentire ad un impiegato di rimuovere i prodotti esistenti nel catalogo.
4. Di ogni prodotto si vuole memorizzare il codice, il nome, la descrizione, la categoria, il prezzo e la quantità.
5. Il sistema deve consentire ad un impiegato di inserire i dati dei fattorini.
6. Di ogni fattorino si vuole memorizzare l'ID, il nome, il cognome, l'e-mail ed il numero di telefono.
7. Il sistema deve offrire al cliente la funzionalità di registrazione.
8. Di ogni cliente registrato si vuole memorizzare il nome utente, la password, l'indirizzo di spedizione, il numero di telefono mobile e il numero della carta di credito.
9. Il sistema deve offrire al cliente la funzionalità di visualizzazione del catalogo dei prodotti.
10. Il sistema deve offrire al cliente registrato la funzionalità di autenticazione nel sistema.
11. Il sistema deve offrire al cliente registrato la funzionalità di visualizzazione del catalogo dei prodotti.
12. Il sistema deve consentire al cliente registrato di effettuare un ordine.
13. Per effettuare l'ordine, il cliente registrato deve specificare nome utente, password, il prodotto e la quantità desiderata.
14. Per effettuare l'ordine, il sistema deve consentire al cliente registrato di richiedere la consegna a domicilio, confermando l'indirizzo di spedizione registrato.
15. Per effettuare l'ordine, il sistema deve consentire al cliente registrato di eseguire il pagamento.
16. Il sistema deve gestire il pagamento tramite l'interfacciamento con un sistema di pagamento esterno.
17. Al completamento del pagamento, il sistema deve inviare una notifica al cliente registrato con il riepilogo dell'ordine ed il prezzo pagato.

18. Al completamento del pagamento, il sistema deve inviare una notifica all'impiegato in negozio per la preparazione del pacco.
19. Al completamento del pagamento, il sistema deve inviare una notifica al fattorino per il ritiro e la consegna del pacco.
20. Il sistema deve gestire l'invio della notifica tramite un servizio di messaggistica.
21. Di ogni ordine si vuole memorizzare l'ID, la data, l'ora, il costo totale, lo stato, il prodotto acquistato, la quantità, la data della consegna, l'ora della consegna, il cliente registrato ed il fattorino delegato alla consegna.
22. Gli stati dell'ordine devono essere: In_corso, Confermato, Spedito, Consegnato.
23. Il sistema deve consentire al fattorino di segnalare l'avvenuta consegna.
24. Il sistema deve registrare la data e l'ora dell'effettiva consegna nel momento in cui il fattorino effettua la segnalazione.
25. Il sistema deve offrire al direttore la funzionalità di generazione di un report di marketing, contenente tutti i clienti registrati che hanno effettuato acquisti nel mese precedente la richiesta.
26. Per ogni cliente registrato, nel report di marketing, il sistema deve visualizzare l'importo complessivo speso nel periodo indicato.
27. Il sistema deve offrire al direttore la funzionalità di generazione di un report contenente i prodotti venduti mensilmente, ai fini dell'approvvigionamento.
28. Per ogni prodotto, nel report di approvvigionamento, il sistema deve visualizzare la relativa quantità totale.
29. Il sistema deve offrire al direttore la funzionalità di generazione di un report contenente i tempi medi di consegna per fattorino per mese.
30. Il sistema deve calcolare il tempo di ogni singola consegna come differenza tra l'orario della notifica al fattorino e l'orario dell'effettiva consegna registrata.

2.3 Glossario dei termini

Termine	Descrizione	Sinonimi
Prodotto	Articolo per la cura dei capelli caratterizzato da codice univoco, nome, descrizione, categoria e prezzo.	
Catalogo	Insieme dei prodotti visualizzabili dagli utenti del sistema.	
Impiegato	Soggetto incaricato della manutenzione del catalogo e della preparazione fisica dei pacchi a seguito di un ordine.	
Fattorino	Soggetto incaricato della consegna a domicilio.	
Cliente	Un generico cliente che visualizza il catalogo senza essere registrato.	
Cliente registrato	Un cliente che ha effettuato la procedura di registrazione.	
Ordine	Registrazione di una transazione d'acquisto relativa a un singolo prodotto.	Pacco
Tempo di consegna	Intervallo di tempo calcolato come differenza tra l'orario della notifica inviata al fattorino e l'orario della segnalazione di consegna registrata.	
Stato dell'ordine	Etichetta descrittiva che identifica la fase corrente del ciclo di vita dell'ordine.	
In_corso	L'ordine è stato inizializzato dal sistema. Il sistema attende l'avvenuto pagamento da parte del cliente registrato.	

Termine	Descrizione	Sinonimi
Confermato	Il sistema ha ricevuto la conferma del pagamento da parte del provider esterno. Il sistema ha notificato il cliente e l'impiegato.	
Spedito	Il sistema ha notificato il fattorino della consegna.	
Consegnato	Il fattorino ha segnalato la chiusura dell'ordine tramite il sistema.	

2.4 Classificazione dei requisiti

2.4.1 Requisiti funzionali

ID	Requisito	Origine
RF01	Il sistema deve consentire ad un impiegato di aggiungere nuovi prodotti nel catalogo.	1
RF02	Il sistema deve consentire ad un impiegato di modificare i prodotti esistenti nel catalogo.	2
RF03	Il sistema deve consentire ad un impiegato di rimuovere i prodotti esistenti nel catalogo.	3
RF04	Il sistema deve consentire ad un impiegato di inserire i dati dei fattorini.	5
RF05	Il sistema deve offrire al cliente la funzionalità di registrazione.	7
RF06	Il sistema deve consentire al cliente di visualizzare il catalogo dei prodotti.	9
RF07	Il sistema deve offrire al cliente registrato la funzionalità di autenticazione nel sistema.	10
RF08	Il sistema deve offrire al cliente registrato la funzionalità di visualizzazione del catalogo dei prodotti.	11
RF09	Il sistema deve consentire al cliente registrato di effettuare un ordine.	12, 14
RF10	Il sistema deve consentire al cliente registrato di eseguire un pagamento.	15
RF11	Al completamento del pagamento, il sistema deve inviare una notifica al cliente registrato con il riepilogo dell'ordine ed il prezzo pagato.	17
RF12	Al completamento del pagamento, il sistema deve inviare una notifica all'impiegato in negozio per la preparazione del pacco.	18
RF13	Al completamento del pagamento, il sistema deve inviare una notifica al fattorino per il ritiro e la consegna del pacco.	19
RF14	Il sistema deve consentire al fattorino di segnalare l'avvenuta consegna.	23, 24
RF15	Il sistema deve offrire al direttore la funzionalità di generazione di un report di marketing, contenente tutti i clienti registrati che hanno effettuato acquisti nel mese precedente la richiesta.	25, 26
RF16	Il sistema deve offrire al direttore la funzionalità di generazione di un report contenente i prodotti venduti mensilmente, ai fini dell'approvvigionamento.	27, 28
RF17	Il sistema deve offrire al direttore la funzionalità di generazione di un report contenente i tempi medi di consegna per fattorino per mese.	29, 30

2.4.2 Requisiti sui dati

ID	Requisito	Origine
RD01	Di ogni prodotto si vuole memorizzare il codice, il nome, la descrizione, la categoria, il prezzo e la quantità.	4
RD02	Di ogni fattorino si vuole memorizzare l'ID, il nome, il cognome, l'e-mail ed il numero di telefono.	6
RD03	Di ogni cliente registrato si vuole memorizzare il nome utente, la password, l'indirizzo di spedizione, il numero di telefono mobile e il numero della carta di credito.	8
RD04	Per effettuare l'ordine, il cliente registrato deve specificare nome utente, password, il prodotto e la quantità desiderata.	13

ID	Requisito	Origine
RD05	Di ogni ordine si vuole memorizzare l'ID, la data, l'ora, il costo totale, lo stato, il prodotto acquistato, la quantità, la data della consegna, l'ora della consegna, il cliente registrato ed il fattorino delegato alla consegna.	21
RD06	Gli stati dell'ordine devono essere: In_corso, Confermato, Spedito, Consegnato.	22

2.4.3 Vincoli / Altri requisiti

ID	Requisito	Origine
V01	Il sistema deve gestire il pagamento tramite l'interfacciamento con un sistema di pagamento esterno.	16
V02	Il sistema deve gestire l'invio della notifica tramite un servizio di messaggistica.	20
RNF01	Il sistema deve essere in grado di stabilire una connessione con il gateway di pagamento esterno entro 10 secondi dall'invio della richiesta, nel 95% dei tentativi effettuati.	
RNF02	Il servizio di messaggistica deve garantire la consegna della notifica entro 60 secondi dalla conferma dell'avvenuto pagamento.	

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Cliente
- Cliente Registrato
- Impiegato
- Fattorino
- Direttore

Casi d'uso:

- **UC01:** Inserisci prodotto
- **UC02:** Modifica prodotto
- **UC03:** Elimina prodotto
- **UC04:** Inserisci fattorino
- **UC05:** Visualizza catalogo
- **UC06:** Registrazione
- **UC07:** Acquista prodotto
- **UC08:** Segnalazione consegna
- **UC09:** Genera report
- **UC10:** Genera report di marketing
- **UC11:** Genera report di approvvigionamento
- **UC12:** Genera report dei tempi di consegna

Attori Secondari:

- Servizio di pagamento
- Servizio di messaggistica

Casi d'uso di inclusione:

- **UC13:** Autenticazione
- **UC14:** Pagamento
- **UC15:** Notifica riepilogo
- **UC16:** Notifica preparazione
- **UC17:** Notifica ritiro

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Requisiti corrispondenti
UC01: Inserisci prodotto	Impiegato	-	-	RF01
UC02: Modifica prodotto	Impiegato	-	-	RF02
UC03: Elimina prodotto	Impiegato	-	-	RF03
UC04: Inserisci fattorino	Impiegato	-	-	RF04
UC05: Visualizza catalogo	Cliente, Cliente Registrato	-	-	RF06, RF08
UC06: Registrazione	Cliente	-	-	RF05
UC07: Acquista prodotto	Cliente Registrato	-	Include: UC13, UC14, UC15, UC16, UC17	RF09
UC08: Segnalazione consegna	Fattorino	-	-	RF14
UC09: Genera report	Direttore	-	Generalizzazione di UC10, UC11, UC12	RF15, RF16, RF17
UC10: Genera report di marketing	Direttore	-	-	RF15
UC11: Genera report approvvigionamento	Direttore	-	-	RF16
UC12: Genera report dei tempi di consegna	Direttore	-	-	RF17
UC13: Autenticazione	-	-	Incluso in UC07	RF07
UC14: Pagamento	-	Servizio di pagamento	Incluso in UC07	RF10
UC15: Notifica riepilogo	-	Servizio di messaggistica	Incluso in UC07	RF11
UC16: Notifica preparazione	-	Servizio di messaggistica	Incluso in UC07	RF12
UC17: Notifica ritiro	-	Servizio di messaggistica	Incluso in UC07	RF13

2.5.2 Diagramma dei casi d'uso

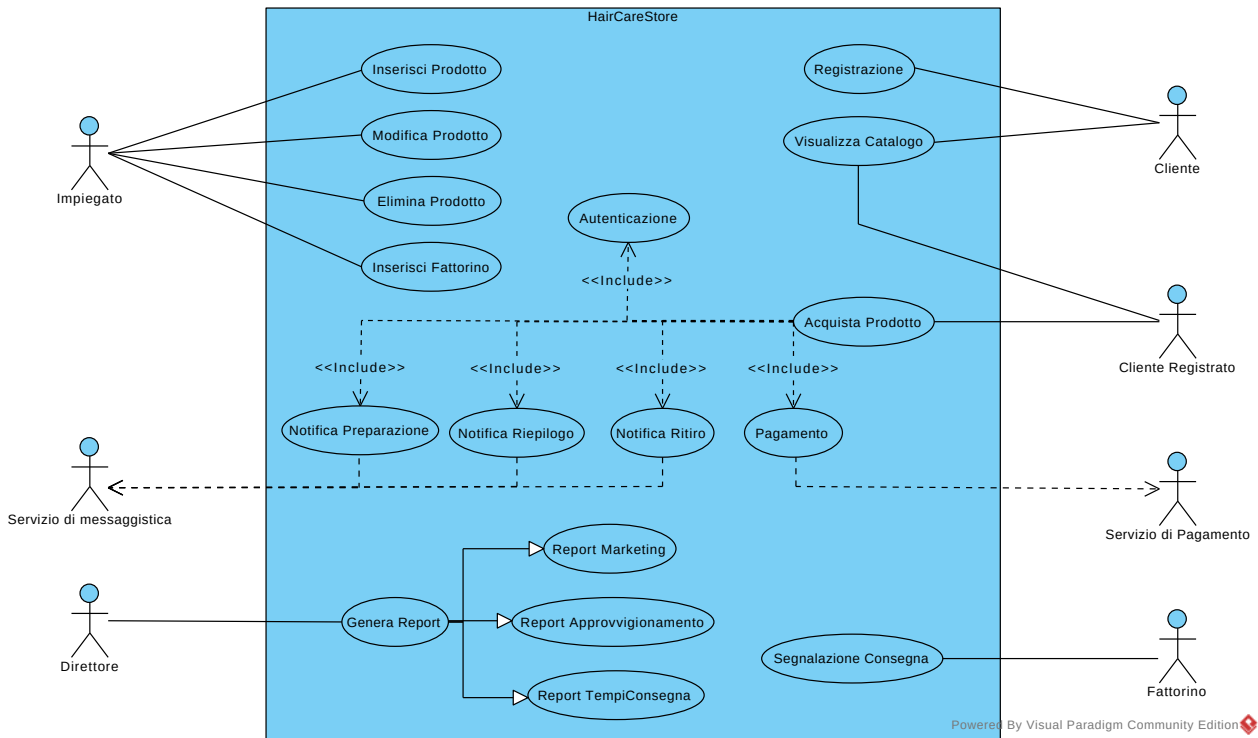


Figura 1: Diagramma dei Casi d'Uso.

2.5.3 Scenari

Caso d'uso	Acquista Prodotto
Attore primario	Cliente Registrato
Attore secondario	Servizio di Pagamento, Servizio di Messaggistica
Descrizione	Un Cliente Registrato acquista un prodotto del catalogo.
Pre-Condizioni	Esistono uno o più prodotti disponibili nel catalogo da poter acquistare ed uno o più fattorini registrati per la consegna dell'ordine.
Sequenza di eventi principale	- Il caso d'uso inizia quando il Cliente registrato richiede l'acquisto di un prodotto dal catalogo. - Il Cliente Registrato inserisce nome utente, password, il prodotto e la quantità desiderata. - Include(Autenticazione) If l'autenticazione non va a buon fine, il sistema restituisce un errore al cliente. - Il sistema controlla che la quantità di prodotto desiderata dal cliente registrato sia disponibile. If il numero di prodotti selezionato non è disponibile viene restituito un errore al cliente registrato.

	4. Il sistema calcola il prezzo totale relativo alla quantità di prodotto selezionato. 5. Il Sistema seleziona il fattorino delegato alla consegna dell'ordine. 6. Il sistema registra l'ordine. 7. Il sistema mostra l'indirizzo di spedizione del cliente registrato. 8. Il sistema mostra il metodo di pagamento del cliente registrato. 9. Il cliente registrato conferma l'ordine. 9.1. If Il cliente registrato non conferma l'ordine, il caso d'uso viene concluso. 10. Include(Pagamento) 10.1. Il Cliente registrato paga il prezzo calcolato. 10.2. Il Sistema riceve la conferma della transazione, eseguita dal servizio di pagamento. 10.3. If Il Sistema riceve una conferma negativa, il caso d'uso viene concluso. 11. Include(Notifica Riepilogo) 11.1. Il Sistema riceve la conferma dell'operazione di notifica, delegata al Servizio di messaggistica. 12. Include(Notifica Preparazione) 12.1. Il Sistema riceve la conferma dell'operazione di notifica, delegata al Servizio di messaggistica. 12.2. Il Sistema modifica lo stato dell'ordine in Confermato. 13. Include(Notifica Ritiro) 13.1. Il Sistema riceve la conferma dell'operazione di notifica, delegata al Servizio di messaggistica. 13.2. Il Sistema modifica lo stato dell'ordine in Spedito.
Post-Condizioni	Il cliente registrato ha acquistato un prodotto e ha ricevuto la notifica di riepilogo dell'ordine. L'ordine è stato memorizzato dal sistema. L'impiegato ha ricevuto la notifica di preparazione dell'ordine. Il Fattorino ha ricevuto la notifica di ritiro dell'ordine.
Casi d'uso correlati	UC13 (Autenticazione), UC14 (Pagamento), UC15 (Notifica riepilogo), UC16 (Notifica preparazione), UC17 (Notifica ritiro)
Sequenza di eventi alternativi	-

2.6 Modellazione dei dati

2.6.1 Progettazione concettuale

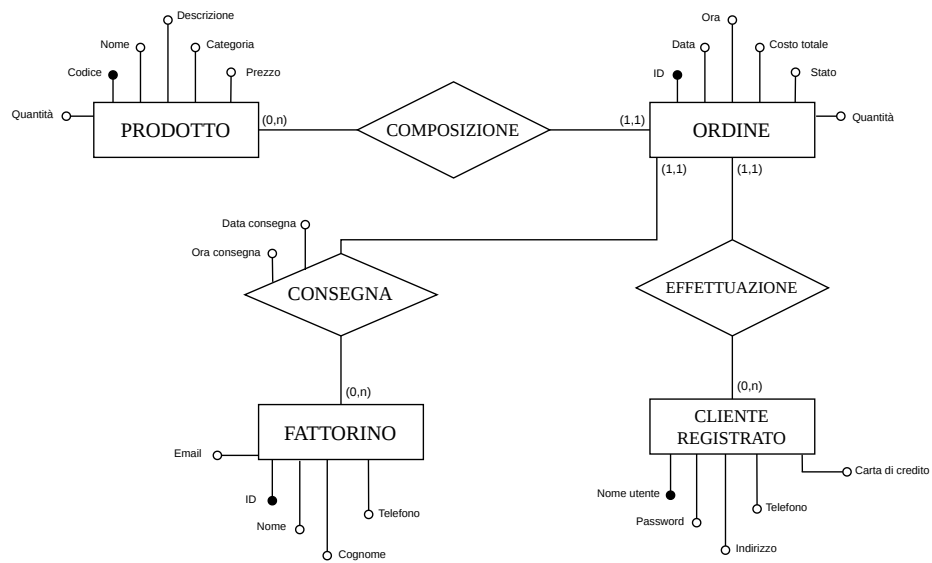


Figura 2: Diagramma E-R

2.7 Diagramma delle classi

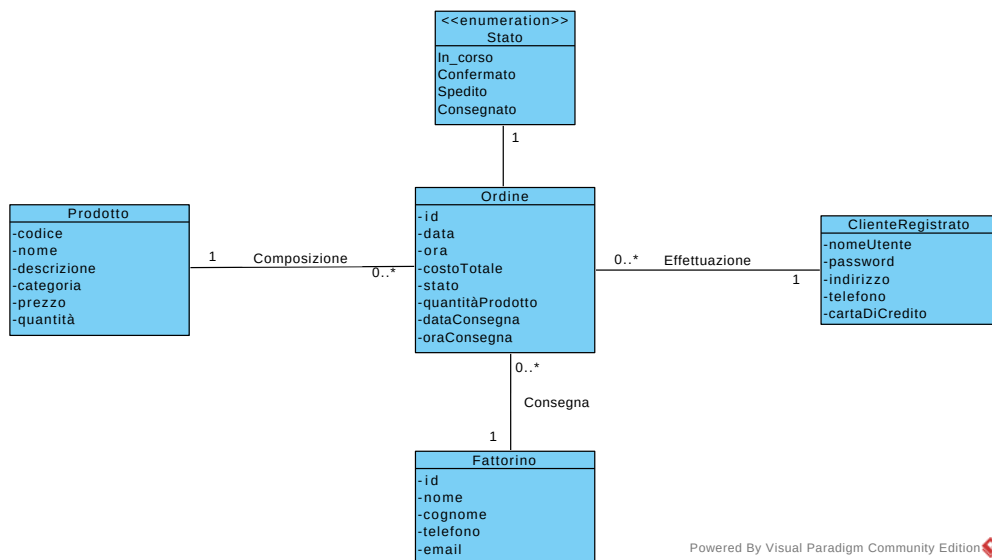


Figura 3: Diagramma delle classi di analisi

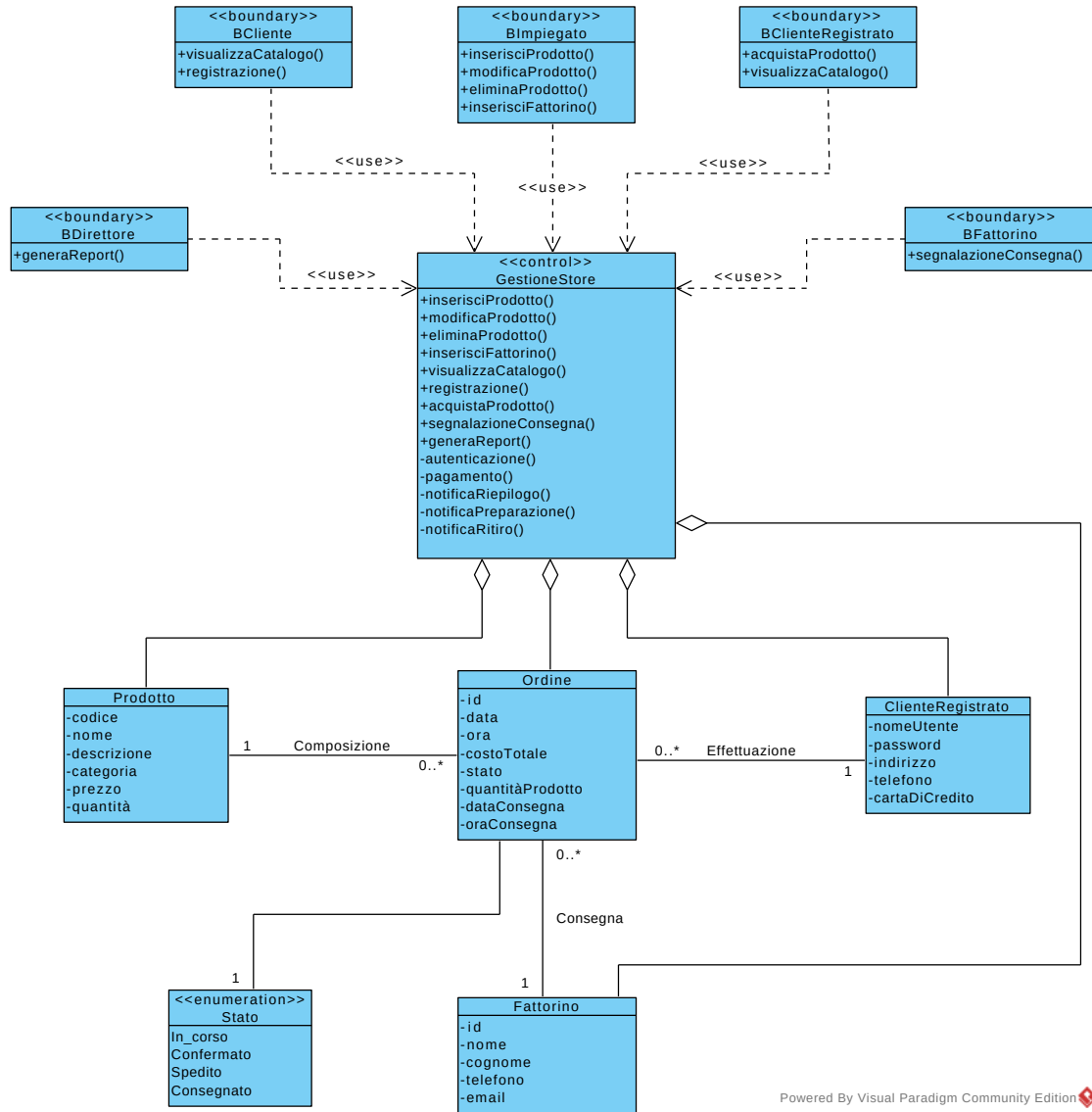


Figura 4: Diagramma delle classi di analisi raffinato (Control e Boundary)

2.8 Diagrammi di sequenza

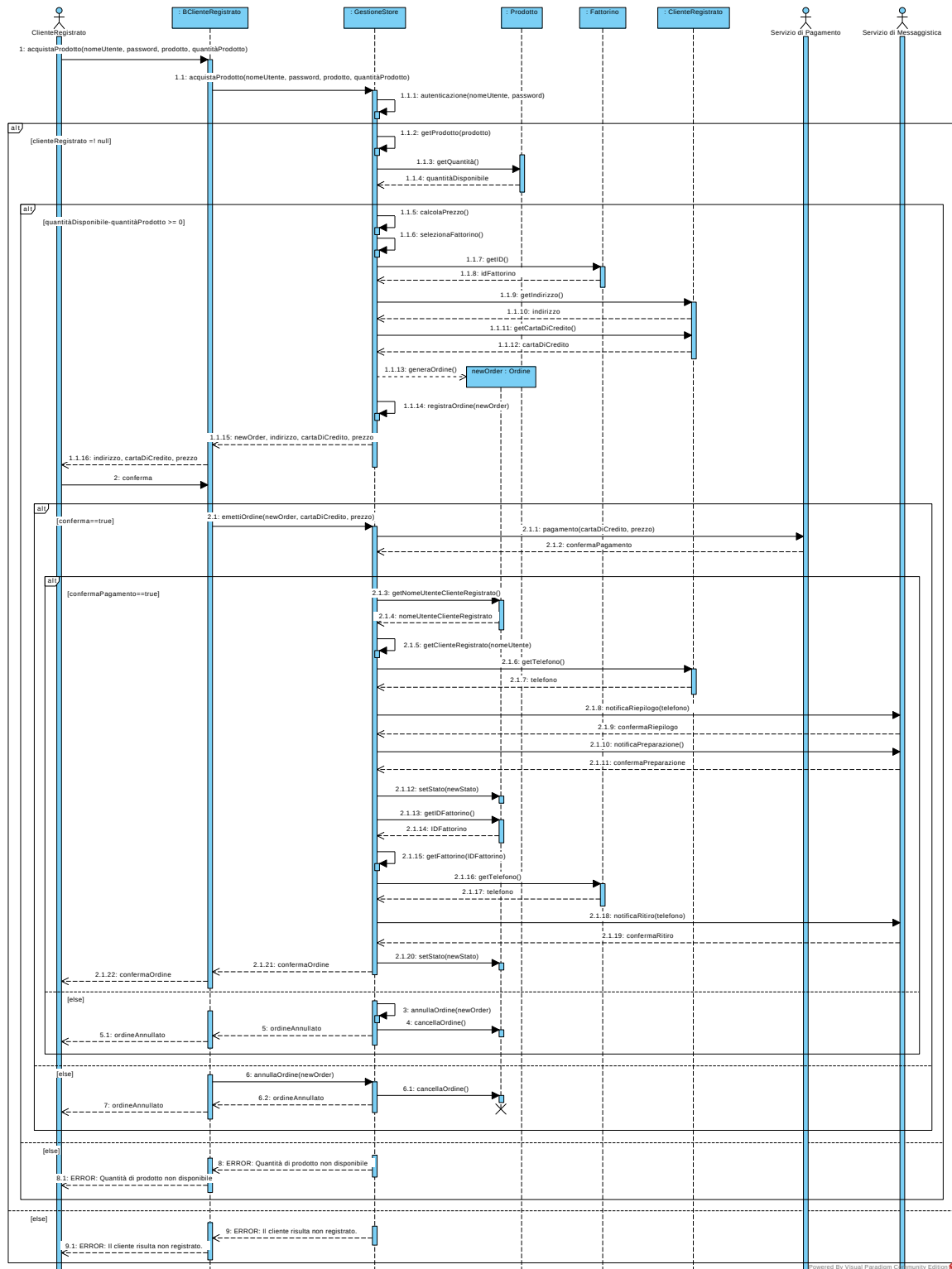


Figura 5: Diagramma di sequenza di analisi

2.9 Verifica della completezza dei requisiti

- **RF01** è modellato nell'UCD con l'attore *Impiegato* e con il caso d'uso UC01.
- **RF02** è modellato nell'UCD con l'attore *Impiegato* e con il caso d'uso UC02.
- **RF03** è modellato nell'UCD con l'attore *Impiegato* e con il caso d'uso UC03.
- **RF04** è modellato nell'UCD con l'attore *Impiegato* e con il caso d'uso UC04.
- **RF05** è modellato nell'UCD con l'attore *Cliente* e con il caso d'uso UC06.
- **RF06** è modellato nell'UCD con l'attore *Cliente* e con il caso d'uso UC05.
- **RF07** è modellato nell'UCD con il caso d'uso UC13.
- **RF08** è modellato nell'UCD con l'attore *Cliente registrato* e con il caso d'uso UC05.
- **RF09** è modellato nell'UCD con l'attore *Cliente registrato* e con il caso d'uso UC07, UC13, UC14, UC15, UC16, UC17.
- **RF10** è modellato nell'UCD con l'attore *Servizio di pagamento* (secondario) e con il caso d'uso UC14.
- **RF11** è modellato nell'UCD con l'attore *Servizio di messaggistica* (secondario) e con il caso d'uso UC15.
- **RF12** è modellato nell'UCD con l'attore *Servizio di messaggistica* (secondario) e con il caso d'uso UC16.
- **RF13** è modellato nell'UCD con l'attore *Servizio di messaggistica* (secondario) e con il caso d'uso UC17.
- **RF14** è modellato nell'UCD con l'attore *Fattorino* e con il caso d'uso UC08.
- **RF15** è modellato nell'UCD con l'attore *Direttore* e con il caso d'uso UC09, UC10.
- **RF16** è modellato nell'UCD con l'attore *Direttore* e con il caso d'uso UC09, UC11.
- **RF17** è modellato nell'UCD con l'attore *Direttore* e con il caso d'uso UC09, UC12.
- **RD01** è modellato nel CD con la classe *Prodotto*.
- **RD02** è modellato nel CD con la classe *Fattorino*.
- **RD03** è modellato nel CD con la classe *ClienteRegistrato*.
- **RD04** è modellato nel SD con i parametri della funzione *acquistaProdotto*.
- **RD05** è modellato nel CD con la classe *Ordine*.
- **RD06** è modellato nel CD con il tipo enumerativo *Stato*.

Leggenda:

- UCD: Use Case Diagram
- CD: Class Diagram
- SD: Sequence Diagram

3 Stima dei costi

Nell'identificazione delle funzioni dati, per ciascun ILF ed EIF, si assegna un valore di complessità funzionale usando la seguente tabella di riferimento:

ILF/EIF	1-19 DET	20-50 DET	51 o più DET
1 RET	bassa	bassa	media
2-5 RET	bassa	media	alta
6 o più RET	media	alta	alta

Tipo	Descrizione	RET	DET	Complessità
ILF	Prodotto	1	6	bassa
ILF	Cliente Registrato	1	5	bassa
ILF	Fattorino	1	5	bassa
ILF	Ordine	1	11	bassa

Nell'identificazione delle funzioni transazionali, per ciascun EI, EO ed EQ, si assegna un valore di complessità funzionale usando la seguenti tabelle di riferimento:

EI	1-4 DET	5-15 DET	16 o più DET
0-1 FTR	bassa	bassa	media
2 FTR	bassa	media	alta
3-4 o più FTR	media	alta	alta

EO/EQ	1-5 DET	6-19 DET	20 o più DET
0-1 FTR	bassa	bassa	media
2-3 FTR	bassa	media	alta
4 o più FTR	media	alta	alta

Tipo	Descrizione	FTR	DET	Elementi	Complessità
EI	Acquisto Prodotto	4	13	• 4 input del Cliente Registrato (Nome Utente, Password, Prodotto, Quantità). • 3 Dati di riepilogo (Costo Totale, Indirizzo, Carta di Credito). • 1 Conferma acquisto. • 1 Messaggio di errore. • 1 Messaggio di conferma.	alta
EO	Notifica Riepilogo	1	6	• 6 output di Ordine.	bassa
EO	Notifica Preparazione	1	6	• 6 output di Ordine.	bassa
EO	Notifica Consegna	2	3	• 2 output di Cliente Registrato (Nome Utente, Indirizzo). • 1 output di Ordine (ID Ordine).	bassa

I valori precedentemente conteggiati sono pesati secondo la tabella seguente:

	SEMPLICE	MEDIO	COMPLESSO
NILF	7	10	15
NEIF	5	7	10
NEI	3	4	6
NEO	4	5	7
NEQ	4	4	6

Calcolo UFP	Complessità			Totale
Funzioni	bassa	media	alta	
Input esterni (EI)	$\dots \times 3$	$\dots \times 4$	1×6	6
Interrogazioni esterne (EQ)	$\dots \times 4$	$\dots \times 4$	$\dots \times 6$	0
Output esterni (EO)	3×4	$\dots \times 5$	$\dots \times 7$	12
File esterni di interfaccia (EIF)	$\dots \times 5$	$\dots \times 7$	$\dots \times 10$	0
File interni logici (ILF)	4×7	$\dots \times 10$	$\dots \times 15$	28
TOTALE UFP				46

Il conteggio UFP è raffinato secondo i 14 fattori correttivi riportati in tabella:

Caratteristiche generali	Valore
Comunicazione dati	3
Distribuzione elaborazione	0
Prestazioni	3
Utilizzo intensivo configurazione	2
Frequenza delle Transazioni	3
Inserimento dati interattivo	4
Efficienza per l'utente finale	4
Aggiornamento interattivo	3
Complessità elaborativa	0
Riusabilità	3
Facilità installazione	2
Facilità gestione operativa	2
Molteplicità di siti	0
Facilità di modifica	3
	TOT: 32

Il valore finale del conteggio FP è:

$$FP = UFP \times \left(0.65 + 0.01 \times \sum_{i=1}^{14} F_i \right) = 44,62$$

Il numero stimato di linee di codice Java è:

$$JAVA = FP \times (LLOC/FP) = 45 \times 53 = 2385$$

4 Piano di test funzionale

Categoria	Classe di Equivalenza
Nome Utente	- Stringa sintatticamente valida (caratteri alfanumerici con lunghezza ≤ 20 caratteri). - Stringa alfanumerica con lunghezza > 20 caratteri. [ERROR] - Stringa contenente simboli che non sono caratteri alfanumerici. [ERROR] - Stringa vuota. [ERROR]
Password	- Stringa sintatticamente valida (caratteri con $8 \leq \text{lunghezza} \leq 20$ caratteri e contenente almeno un carattere speciale). - Stringa con lunghezza non valida (< 8 caratteri o > 20 caratteri). [ERROR] - Stringa non contenente almeno un carattere speciale. [ERROR] - Stringa vuota. [ERROR]
Prodotto	- Stringa sintatticamente valida (caratteri alfanumerici con lunghezza $= 15$ caratteri ed incluso il simbolo " - "). - Stringa con lunghezza non valida (lunghezza $\neq 15$ caratteri). [ERROR] - Stringa contenente simboli che non sono caratteri alfanumerici. [ERROR] - Stringa vuota. [ERROR]
Quantità	- Valore intero > 0. - Valore intero $= 0$ [ERROR] - Valore intero < 0. [ERROR]
Conferma Ordine	- Cliente Registrato conferma l'ordine. - Cliente annulla l'ordine. [SINGLE]
Database: Cliente Registrato	- Corrispondenza corretta tra Nome Utente e Password. - Nome Utente non presente. [ERROR] - Password inserita non corrispondente al Nome Utente. [ERROR]
Database: Prodotto	- Quantità disponibile $\geq$ quantità selezionata. - Quantità disponibile $<$ quantità selezionata. [ERROR]

Sono presenti 7 categorie di cui:

- **3** con 4 classi di equivalenza;
- **2** con 3 classi di equivalenza;
- **2** con 2 classi di equivalenza.

Il numero di **Test** da effettuare senza considerare i vincoli è:

$$\text{TEST} = 4^3 \times 3^2 \times 2^2 = 2304$$

Il numero di **Test** per testare singolarmente i vincoli [ERROR] e [SINGLE] è:

$$\text{TEST}_{\text{VINCOLI}} = 3 + 3 + 3 + 2 + 1 + 2 + 1 = 15$$

Il numero di **Test risultante** è:

$$\text{TEST}_{\text{RIS}} = (1 \times 1 \times 1 \times 1 \times 1 \times 1 \times 1) + 15 = \mathbf{16}$$

4.1 Test Suite

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
TC01	Acquisto completato con successo con tutti gli input validi e corretti.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Prodotto esistente e disponibile nel Database. Almeno un fattorino disponibile nel Database.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Calcolo prezzo totale. Mostra indirizzo e metodo di pagamento. Mostra conferma dell'ordine.	Invio notifiche a cliente registrato, impiegato e fattorino. Ordine memorizzato e quantità di prodotto disponibile aggiornata.
TC02	Inserimento di un nome utente che supera i 20 caratteri limite.	- Nome Utente > 20 caratteri [ERROR]; - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento del nome utente.	{Nome Utente : "andreavisconti1234567", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato nome utente non valido, troppo lungo!	Il sistema richiede di inserire nuovamente il nome utente.
TC03	Inserimento di un nome utente contenente simboli non alfanumerici.	- Nome Utente con simboli non alfanumerici [ERROR]; - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento del nome utente.	{Nome Utente : "Hendrix_", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato nome utente non valido, contiene caratteri non consentiti!	Il sistema richiede di inserire nuovamente il nome utente.

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
TC04	Inserimento di un nome utente vuoto.	- Nome Utente vuoto [ERROR]; - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento del nome utente.	{Nome Utente : "", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Inserire un Nome Utente!	Il sistema richiede di inserire nuovamente il nome utente.
TC05	Inserimento di una password con lunghezza non valida (numero di caratteri < 8 o > 20).	- Nome Utente (valido); - Password < 8 o > 20 caratteri [ERROR]; - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della password.	{Nome Utente : "Hendrix", Password : "N4600!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato password non valido!	Il sistema richiede di inserire nuovamente la password.
TC06	Inserimento di una password mancante di almeno un carattere speciale.	- Nome Utente (valido); - Password priva di almeno un carattere speciale [ERROR]; - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della password.	{Nome Utente : "Hendrix", Password : "N46007557", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato password non valido, carattere speciale mancante!	Il sistema richiede di inserire nuovamente la password.

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
TC07	Inserimento di una password vuota.	• Nome Utente (valido); • Password vuota [ERROR]; • Prodotto (valido); • Quantità (valido); • Conferma Ordine (valido); • DB Cliente (valido); • DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della password.	{Nome Utente : "Hendrix", Password : "", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Inserire la password!	Il sistema richiede di inserire nuovamente la password.
TC08	Codice prodotto con una lunghezza diversa da 15 caratteri.	• Nome Utente (valido); • Password (valido); • Prodotto con lunghezza $\neq$ 15 caratteri [ERROR]; • Quantità (valido); • Conferma Ordine (valido); • DB Cliente (valido); • DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato il prodotto da acquistare.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY1", Quantità : 2, Conferma Ordine : True}	Prodotto non valido, lunghezza non valida!	Il sistema richiede di inserire nuovamente il prodotto.
TC09	Codice prodotto contenente simboli non alfanumerici.	• Nome Utente (valido); • Password (valido); • Prodotto con simboli non alfanumerici [ERROR]; • Quantità (valido); • Conferma Ordine (valido); • DB Cliente (valido); • DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato il prodotto da acquistare.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DR?", Quantità : 2, Conferma Ordine : True}	Prodotto non valido, formattazione errata!	Il sistema richiede di inserire nuovamente il prodotto.

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
TC10	Codice prodotto vuoto.	- Nome Utente (valido); - Password (valido); - Prodotto vuoto[ERROR]; - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato il prodotto da acquistare.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "", Quantità : 2, Conferma Ordine : True}	Inserire un Prodotto!	Il sistema richiede di inserire nuovamente il prodotto.
TC11	Inserimento di una quantità richiesta pari a 0.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità = 0 [ERROR]; - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della quantità.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 0, Conferma Ordine : True}	Inserire una quantità positiva!	Il sistema richiede di inserire nuovamente la quantità.
TC12	Inserimento di una quantità richiesta minore di 0.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità < 0 [ERROR]; - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della quantità.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : -1, Conferma Ordine : True}	Inserire una quantità positiva!	Il sistema richiede di inserire nuovamente la quantità.

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
TC13	Il cliente registrato non conferma l'ordine.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine = false [ERROR]; - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database con quantità disponibile sufficiente. Cliente Registrato esistente nel Database. Il sistema richiede la conferma dell'ordine.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : False}	Acquisto annullato!	L'ordine viene cancellato dal Database.
TC14	Autenticazione fallita per nome utente non registrato nel Database.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente Nome Utente non presente [ERROR]; - DB Prodotto (valido).	Cliente Registrato non esistente nel Database.	{Nome Utente : "Hhendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Autenticazione fallita. Nome Utente o Password errati!	Il sistema richiede di inserire nuovamente il Nome Utente.
TC15	Autenticazione fallita per password errata, non corrispondente al nome utente.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente Password errata [ERROR]; - DB Prodotto (valido).	Cliente Registrato non esistente nel Database.	{Nome Utente : "Hendrix", Password : "N4600755!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Autenticazione fallita. Nome Utente o Password errati!	Il sistema richiede di inserire nuovamente la password.

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
TC16	Tentativo di acquistare più unità di quelle disponibili sul Database.	• Nome Utente (valido); • Password (valido); • Prodotto (valido); • Quantità (valido); • Conferma Ordine (valido); • DB Cliente (valido); • DB Prodotto quantità disponibile < quantità selezionata [ERROR].	Esiste una sola unità di prodotto disponibile.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Quantità di prodotto non disponibile!	Il sistema richiede di inserire nuovamente la quantità.

5 Progettazione

5.1 Progettazione della base di dati

Schema Relazionale

PRODOTTI (codice, nome, descrizione, categoria, prezzo, quantità)
FATTORINI (id, nome, cognome, telefono, email)
CLIENTI_REGISTRATI (nome_utente, password, indirizzo, telefono, carta_di_credito)
ORDINI (id, data, ora, costo_totale, stato, quantità, data_consegna, ora_consegna, prodotto:PRODOTTI, fattorino:FATTORINI, cliente_registrato:CLIENTI_REGISTRATI)

Create Table Prodotti

```
CREATE TABLE PRODOTTI (  
    codice CHAR(15),  
    nome VARCHAR(50) NOT NULL,  
    descrizione VARCHAR(100),  
    categoria VARCHAR(15),  
    prezzo DECIMAL(6,2) NOT NULL,  
    quantità INT NOT NULL,  
    CONSTRAINT PK_PRODOTTI PRIMARY KEY (codice),  
    CONSTRAINT CHK_PREZZO CHECK (prezzo>=0),  
    CONSTRAINT CHK_QUANTITA CHECK (quantita>=0),  
    CONSTRAINT CHK_CODICE CHECK (codice REGEXP '~[a-zA-Z0-9-]+$')  
);
```

Create Table Fattorini

```
CREATE TABLE FATTORINI (  
    id INT AUTO_INCREMENT,  
    nome VARCHAR(20) NOT NULL,  
    cognome VARCHAR(20) NOT NULL,  
    telefono VARCHAR(12) NOT NULL,  
    email VARCHAR(50),  
    CONSTRAINT PK_FATTORINI PRIMARY KEY (id)  
);
```

Create Table Clienti_Registrati

```
CREATE TABLE CLIENTI_REGISTRATI (  
    nome_utente VARCHAR(20),  
    password VARCHAR(20) NOT NULL,  
    indirizzo VARCHAR(50) NOT NULL,  
    telefono VARCHAR(12) NOT NULL,  
    carta_di_credito VARCHAR(16) NOT NULL,  
    CONSTRAINT PK_CLIENTI_REGISTRATI PRIMARY KEY (nome_utente),  
    CONSTRAINT CHK_NOME_UTENTE CHECK (nome_utente REGEXP '~[a-zA-Z0-9]+$'),  
    CONSTRAINT CHK_PASSWORD CHECK (LENGTH(password) BETWEEN 8 AND 20  
        AND password REGEXP '~[a-zA-Z0-9-]+$')  
);
```


Create Table Ordini

```
CREATE TABLE ORDINI (
  id INT AUTO_INCREMENT,
  data DATE NOT NULL,
  ora TIME NOT NULL,
  costo_totale DECIMAL (10,2) NOT NULL,
  stato ENUM('In_corso', 'Confermato', 'Spedito', 'Consegnato') NOT NULL DEFAULT
  ↪ 'In_corso'
  quantità INT NOT NULL,
  data_consegna DATE DEFAULT NULL,
  ora_consegna TIME DEFAULT NULL,
  prodotto CHAR(15) NOT NULL,
  fattorino INT NOT NULL,
  cliente_registrato VARCHAR(20) NOT NULL,
  CONSTRAINT PK_ORDINI PRIMARY KEY (id),
  CONSTRAINT CHK_QUANTITA CHECK (quantità>0),
  CONSTRAINT FK_PRODOTTO FOREIGN KEY (prodotto) REFERENCES PRODOTTI(codice) ON
  ↪ UPDATE CASCADE ON DELETE NO ACTION,
  CONSTRAINT FK_FATTORINO FOREIGN KEY (fattorino) REFERENCES FATTORINI(id) ON
  ↪ UPDATE CASCADE ON DELETE NO ACTION,
  CONSTRAINT FK_CLIENTI_REGISTRATI FOREIGN KEY (cliente_registrato) REFERENCES
  ↪ CLIENTI_REGISTRATI(nome_utente) ON UPDATE CASCADE ON DELETE NO ACTION
);
```

5.2 Diagramma delle classi

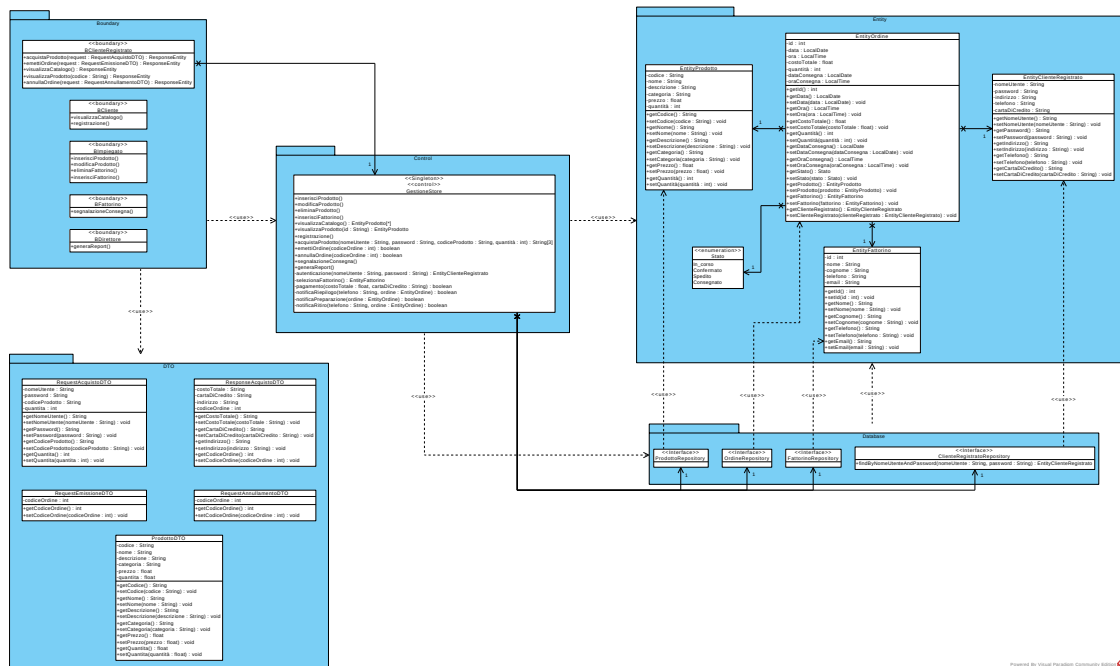


Figura 6: Diagramma delle classi di progetto

5.3 Diagrammi di sequenza

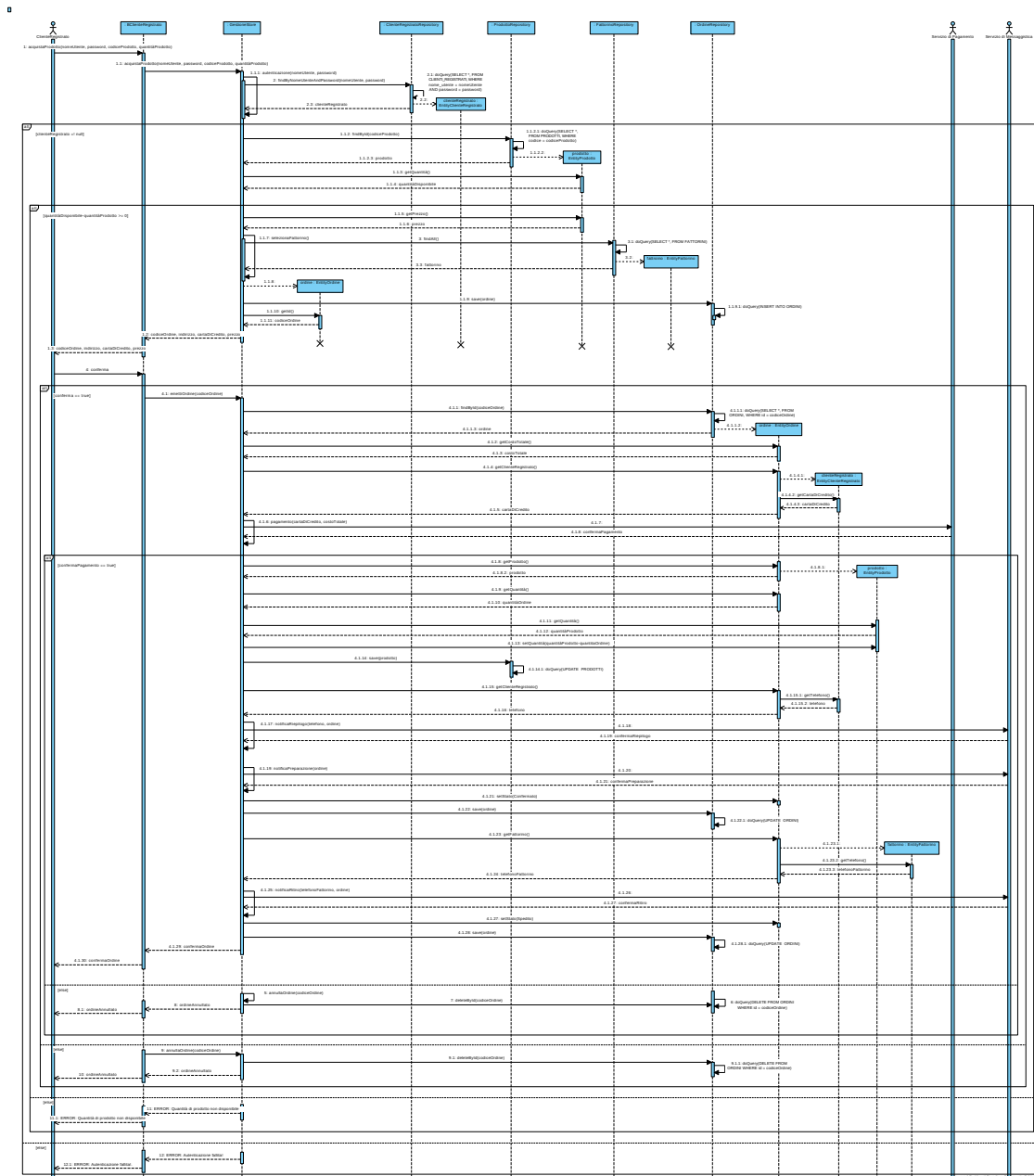


Figura 7: Diagramma di sequenza di progetto

6 Implementazione

In questa sezione viene descritta dettagliatamente l'architettura di implementazione del software *HairCareStore*. Vengono illustrati gli artefatti di codifica prodotti, l'organizzazione dei package, i requisiti d'ambiente per l'installazione e l'esecuzione del software, le metriche di dimensionamento espresse in linee di codice (LOC/LLOC) e, infine, l'analisi degli scostamenti rispetto alle stime dei costi condotte in fase di analisi tramite Function Point Analysis.

L'implementazione dell'applicazione segue i principi dell'architettura *Boundary-Control-Entity*. Nello specifico, è stata sviluppata un'applicazione web garantendo un forte disaccoppiamento tra lo strato di backend e quello di frontend. Lo strato di backend, dedicato alla logica di business e all'accesso ai dati, è realizzato tramite il framework Spring Boot ed il linguaggio di programmazione orientato agli oggetti Java. Lo strato di frontend, dedicato all'interfacciamento con l'utente, è invece sviluppato tramite il framework Angular, basato su TypeScript. I due strati comunicano tramite un'API, ovvero un contratto stabile tra componenti software che evolvono indipendentemente. Nello specifico, è stato utilizzato lo stile architetturale REST al fine di identificare ogni risorsa tramite un URI, ed ogni azione tramite uno specifico metodo HTTP. In particolare, i metodi HTTP mappano direttamente le classiche operazioni CRUD sulla risorsa:

- **POST:** Crea una nuova risorsa.
- **GET:** Legge una risorsa.
- **PUT:** Aggiorna una risorsa.
- **DELETE:** Elimina una risorsa.

Lo scambio di messaggi tra i due strati avviene tramite formato JSON.

6.1 Organizzazione dei package - Backend

Di seguito viene dettagliata la responsabilità di ciascun package per l'applicazione Java di backend, strutturato all'interno della directory `src/main/java`:

Boundary: Rappresenta lo strato di interfaccia verso l'esterno. Contiene i REST Controller deputati all'esposizione degli endpoint HTTP e alla ricezione delle richieste provenienti dal frontend Angular. La classe `BClienteRegistrato.java` intercetta e valida le chiamate relative alle operazioni eseguibili dall'utente registrato.

Control: Costituisce il nucleo algoritmico del sistema, dove risiede la logica di business. All'interno del package, la classe `GestioneStore.java` implementa i casi d'uso, coordinando il flusso di dati tra lo strato di Boundary e lo strato dati.

Entity: Modella il dominio dell'applicazione e definisce la struttura dei dati persistenti. Classi come `EntityClienteRegistrato.java`, `EntityOrdine.java` e `EntityProdotto.java`, insieme all'enumerazione `Stato.java`, mappano direttamente le tabelle del database relazionale tramite le annotazioni dell'Object-Relational Mapping fornito da JPA.

Database: Contiene le interfacce di persistenza che estendono le specifiche di Spring Data JPA. Questo strato agisce da Data Access Object, astruendo completamente la gestione delle query SQL e consentendo le operazioni di lettura e scrittura sul database in modo sicuro e tipizzato.

DTO: Ospita gli oggetti Data Transfer Object adibiti al trasferimento dei dati tra lo strato di frontend e quello di backend. Oltre a garantire un forte disaccoppiamento ed evitare l'esposizione diretta delle entità di persistenza, i DTO adibiti alle richieste integrano la logica di controllo e validazione formale dei dati in input. Tramite apposite annotazioni di *validation*, essi assicurano che i parametri inviati dal frontend rispettino i vincoli sintattici e di integrità prima che la richiesta venga presa in carico dallo strato di Control.

Exception: Raccoglie le eccezioni definite per il dominio dell'applicazione. Questo approccio consente una gestione centralizzata degli errori, permettendo al backend di intercettare i fallimenti applicativi o di database e tradurli in codici di stato HTTP semanticamente corretti per il client.

6.2 Configurazione Backend

La configurazione del modulo di backend si articola principalmente su due file fondamentali posizionati all'interno della struttura del progetto: il file `pom.xml`, deputato alla gestione della build e delle dipendenze, e il file `application.properties`, mirato alla definizione dei parametri di runtime e di connettività.

6.2.1 `pom.xml`

Collocato alla radice del modulo, il file `pom.xml` (*Project Object Model*) è l'artefatto fondamentale per la gestione del ciclo di vita del software tramite lo strumento di build automation Apache Maven. Questo file descrive in modo dichiarativo la struttura del progetto, sollevando lo sviluppatore dalla gestione manuale delle librerie esterne. All'interno del file `pom.xml` del progetto sono definiti i seguenti elementi:

- **Coordinate del Progetto:** Identificano univocamente il modulo all'interno dell'ecosistema Maven tramite i tag `groupId` (lo spazio dei nomi dell'organizzazione), `artifactId` (il nome specifico del software, ovvero `haircarestore_backend`) e `version`.
- **Ereditarietà:** Il progetto eredita le configurazioni di base dal `spring-boot-starter-parent`. Questo modulo centralizza la gestione delle versioni delle dipendenze, garantendo la totale compatibilità tra le librerie ed evitando conflitti a tempo di compilazione.
- **Proprietà d'Ambiente:** Definiscono i requisiti software del sistema, specificando in questo caso l'utilizzo dell'ambiente di runtime **Java 25**.
- **Gestione delle Dipendenze:** Elenca i framework e i driver esterni che Maven deve scaricare automaticamente dai repository centrali per permettere il funzionamento del software. Nello specifico, vengono importati gli starter per lo sviluppo web (`WebMVC` e server embedded Tomcat), per la persistenza dei dati (`Data JPA`), per il driver di connessione a `MySQL` e per il motore di validazione formale dell'input sui DTO (`Validation`).
- **Automazione della Build:** Viene configurato lo `spring-boot-maven-plugin`, il cui scopo è intercettare la fase di packaging per comprimere tutto il codice compilato e le relative dipendenze all'interno di un unico file `.jar` autonomo ed eseguibile. In questo modo, l'applicazione può essere avviata in qualsiasi ambiente di produzione tramite una singola istruzione da terminale (`java -jar`), senza dipendere da un ambiente di sviluppo.

6.2.2 `application.properties`

Collocato all'interno della cartella `src/main/resources`, il file `application.properties` viene analizzato automaticamente da Spring Boot all'avvio per inizializzare i componenti del sistema. Le direttive di configurazione definite per il software sono:

- **Connessione al Database:** Definisce i parametri di rete e di sicurezza necessari per la comunicazione con il DBMS tramite la specifica JDBC. La proprietà `spring.datasource.url` specifica il protocollo, l'host locale (`localhost`), la porta (`3306`) e lo schema relazionale (`hairCareStoreDB`), completati dal driver ufficiale (`com.mysql.cj.jdbc.Driver`) e dalle credenziali di autenticazione (`username` e `password`).
- **Configurazione dell'ORM (JPA/Hibernate):**
 - `spring.jpa.database-platform`: Configura il dialetto SQL specifico (`MySQLDialect`), garantendo che l'ORM generi a runtime query per `MySQL`.
 - `spring.jpa.hibernate.ddl-auto`: Impostata su `none`, vieta a Hibernate di manipolare lo schema del database all'avvio, delegando la creazione delle tabelle a script di inizializzazione, nel nostro caso è presente nel progetto il file di testo (`DB_init.txt`).
 - `spring.jpa.hibernate.naming.physical-strategy`: Forza il framework a mappare i nomi di tabelle e colonne rispettando l'esatta nomenclatura definita nelle classi `Entity`, disabilitando le conversioni automatiche.

6.3 Organizzazione dei package - Frontend

Di seguito viene dettagliata l'organizzazione dei componenti principali presenti nel progetto Angular per il frontend:

public/: La directory è dedicata alla memorizzazione delle risorse statiche globali dell'applicazione, accessibili direttamente dal browser. Nel nostro caso, ospita gli elementi grafici dell'interfaccia utente come il logo del prodotto software (`logo.png`), lo sfondo (`Background.png`) e le immagini rappresentative dei prodotti del catalogo (`Shampoo.png`, `Olio.png`, `Maschera.png`).

src/app/components/: Contiene i componenti grafici riutilizzabili e modulari che compongono l'interfaccia. Ciascun componente è organizzato in una sotto-cartella specifica per isolare logicamente i singoli casi d'uso (`acquisto`, `prodotto`, `prodotto-list`) ed è rigorosamente suddiviso in una triade di file:

- File `.html`: Definisce la struttura della pagina web.
- File `.css`: Definisce le regole di stile locali per la formattazione grafica.
- File `.ts`: Implementa il codice TypeScript preposto alla gestione del comportamento del componente e all'interazione con l'utente.

src/app/models/: Ospita le classi TypeScript (`prodotto.ts`, `acquisto.ts`) adibite alla modellazione dei dati all'interno del frontend. Questi modelli definiscono le strutture dati che l'applicazione si aspetta di manipolare, riflettendo in modo speculare i campi definiti nei DTO del backend e garantendo l'integrità dei dati scambiati.

src/app/services/: Contiene lo strato dei servizi, agendo come intermediario tra i componenti grafici ed il backend. Utilizzando il client HTTP nativo di Angular, il servizio `hairCareStore.service.ts` realizza le chiamate asincrone verso gli endpoint REST del server Spring Boot, isolando la logica di rete e di recupero dati dalla UI.

6.4 Configurazione Frontend

La definizione della *Single Page Application* è delegata un insieme di file posizionati nella radice della cartella `app`:

- **app.routes.ts:** Costituisce il motore di *routing* del frontend. In questo file vengono mappati i percorsi degli URL del browser ai rispettivi componenti grafici. Questo meccanismo permette una navigazione fluida e istantanea, aggiornando dinamicamente porzioni di pagina senza richiedere il ricaricamento dell'intera applicazione.
- **app.config.ts:** Rappresenta il punto di configurazione centralizzato per l'architettura Angular.
- **app.ts:** Identifica la classe del componente radice del frontend. Contiene la logica TypeScript principale di aggancio al framework ed esegue il bootstrap dell'applicazione, orchestrando il ciclo di vita iniziale dell'interfaccia.
- **app.html:** Questo file definisce il layout statico dell'applicazione e ospita la direttiva di reindirizzamento `<router-outlet>` per sostituire dinamicamente i template dei vari sottocomponenti in base alla rotta attiva.
- **app.css:** Custodisce i fogli di stile e le regole di formattazione grafica applicate a livello del componente radice. Viene utilizzato per impostare le proprietà geometriche e i vincoli visivi globali del contenitore principale del software, garantendo la coerenza visiva dell'applicazione prima dell'attivazione delle singole viste interne.

6.5 Database

Il corretto funzionamento del backend di è strettamente subordinato alla disponibilità del Database Management System relazionale. Nel contesto d'esecuzione software, la persistenza dei dati è affidata a **MySQL**, gestito ed eseguito tramite la suite software **XAMPP**. Per consentire l'avvio e l'esecuzione corretta dell'applicazione senza errori di connettività, è necessario rispettare i seguenti passi operativi:

1. **Attivazione del Servizio:** Tramite il pannello di controllo di XAMPP, è necessario avviare il modulo *MySQL*, il quale si pone in ascolto sulla porta standard 3306 locale (*localhost*), esponendo il driver JDBC necessario al backend.
2. **Creazione dello Schema:** Poiché l'ORM del backend è configurato per non manipolare o generare autonomamente la struttura del database all'avvio, l'istanza dello schema relazionale deve essere pre-esistente. È pertanto richiesta la creazione di un database denominato **hairCareStoreDB**.
3. **Popolamento e DDL:** Per definire i vincoli di integrità referenziale, le tabelle e il set di dati iniziale, è necessario eseguire sulla console MySQL (o tramite l'interfaccia grafica *phpMyAdmin* inclusa in XAMPP) lo script SQL di inizializzazione presente alla radice del progetto nel file *DB_init.txt*.

Se questo strato infrastrutturale non risulta attivo e correttamente configurato prima del lancio del file *.jar* del backend, il framework Spring Boot interromperà immediatamente il processo di boot sollevando un'eccezione di fallita connessione al data source.

6.6 Documentazione Javadoc

Al fine di garantire la manutenibilità, l'estensibilità e la trasparenza del codice sorgente del modulo di backend, l'intera base di codice è stata documentata utilizzando Javadoc, lo standard ufficiale della piattaforma Java per la generazione di documentazione tecnica. La documentazione è localizzata all'interno della cartella di progetto al percorso **target/reports/apidocs/**. Per consultare l'intera struttura ipertestuale e navigare agevolmente tra i moduli è sufficiente aprire il file *index.html* tramite un qualsiasi browser web. Inoltre, all'interno del presente elaborato, viene allegata nello specifico la sezione di documentazione relativa alle classi dei package **Boundary** e **Control** associate alle funzionalità sviluppate per il sistema.

Class GestioneStore

`java.lang.Object`
`Control.GestioneStore`

```
@Service
public class GestioneStore
extends Object
```

Strato Control della BCED, dedicata all'implementazione della logica di business. All'interno del sistema sviluppato, i componenti strutturali non vengono istanziati mediante l'operatore nativo `new` del linguaggio Java, bensì la loro gestione è interamente delegata al container IoC (Inversion of Control) di Spring Boot. Un oggetto il cui intero ciclo di vita, inclusi i processi di istanziazione, configurazione, assemblaggio e distruzione, è coordinato dal framework prende il nome di Spring Bean. La registrazione dei componenti all'interno del container avviene in modalità dichiarativa sfruttando gli stereotipi standard forniti dal framework. Per poter registrare lo strato Control al container e dichiararla esplicitamente un Bean, utilizziamo l'annotazione:

`@Service`: rende esplicito il ruolo della classe all'interno dell'architettura BCED. Per garantire l'efficienza computazionale e la coerenza architetturale, ad ogni Spring Bean del sistema viene associato lo scope Singleton di default. Sotto questo ciclo di vita, l'IoC Container crea una sola e unica istanza del componente per l'intero ciclo di vita dell'ApplicationContext. Anche i repository non sono istanziati manualmente, ma ci limitiamo a dichiarare i relativi attributi come `final` e li accettiamo come parametri di ingresso nel costruttore della classe. Spring provvede a rintracciare i bean candidati all'interno del proprio "ApplicationContext" e li inietta a runtime.

Constructor Summary

Constructors

Constructor	Description
GestioneStore (<code>ClienteRegistratoRepository</code> clienteRegistratoRepository, <code>FattorinoRepository</code> fattorinoRepository, <code>OrdineRepository</code> ordineRepository, <code>ProdottoRepository</code> prodottoRepository)	Costruttore con argomenti.

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
<code>ArrayList <String ></code>	acquistaProdotto (<code>String</code>	Avvia la prima fase del caso

	<code>nomeUtente, String password, String codiceProdotto, int quantità)</code>	d'uso "Acquista Prodotto" (UCo7).
boolean	<code>annullaOrdine (int codiceOrdine)</code>	Flusso alternativo associato al rifiuto o all'annullamento dell'acquisto da parte dell'utente.
boolean	<code>emettiOrdine (int codiceOrdine)</code>	Avvia la seconda fase del caso d'uso "Acquista Prodotto" (UCo7), in seguito alla conferma dell'ordine.
<code>List <EntityProdotto></code>	<code>visualizzaCatalogo()</code>	Recupera l'elenco completo di tutti i prodotti presenti nel catalogo, mappando il caso d'uso visualizza catalogo (UCo5).
<code>EntityProdotto</code>	<code>visualizzaProdotto(String id)</code>	Recupera il singolo prodotto presente nel catalogo, mappando il caso d'uso visualizza catalogo (UCo5).

Methods inherited from class **Object**

`clone` , `equals` , `finalize` , `getClass` , `hashCode` , `notify` , `notifyAll` ,
`toString` , `wait` , `wait` , `wait`

Constructor Details

GestioneStore

```
public GestioneStore
(ClienteRegistratoRepository clienteRegistratoRepository,
 FattorinoRepository fattorinoRepository,
 OrdineRepository ordineRepository,
 ProdottoRepository prodottoRepository)
```

Costruttore con argomenti. Spring genera automaticamente in memoria una classe concreta che implementa le interfacce Repository. Questa classe generata dal framework, chiamata Proxy, contiene l'effettivo codice SQL per comunicare con il database. Una volta creata l'istanza di questa classe generata automaticamente, Spring la inserisce all'interno dell' IoC Container.

Parameters:

`clienteRegistratoRepository` - Riferimento al cliente registrato

fattorinoRepository - Riferimento al fattorino

ordineRepository - Riferimento all'ordine

prodottoRepository - Riferimento al prodotto

Method Details

acquistaProdotto

```
public ArrayList <String > acquistaProdotto(String nomeUtente,  
                                             String password,  
                                             String codiceProdotto,  
                                             int quantità)  
    throws AutenticazioneFallitaException,  
           RisorsaNonTrovataException,  
           DisponibilitàEsauritaException,  
           DatabaseException
```

Avvia la prima fase del caso d'uso "Acquista Prodotto" (UC07). La struttura dell'algoritmo è la seguente:

1. Autenticazione del Cliente Registrato.
2. Recupero dal Database del Prodotto selezionato.
3. Controllo della quantità disponibile.
4. Calcolo del prezzo totale.
5. Selezione del Fattorino.
6. Costruzione e Salvataggio dell'ordine. Viene restituito in Output alla Boundary un ArrayList. Ricordiamo che la libreria java.util offre un insieme completo di classi contenitore, ovvero oggetti che raggruppano elementi multipli in una singola unità. ArrayList implementa tramite List l'interfaccia Collection, realizzando una raccolta sequenziale di singoli elementi. ArrayList implementa l'interfaccia List, tale per cui può contenere elementi duplicati.

Parameters:

nomeUtente - Username del cliente registrato

password - Password del cliente registrato

codiceProdotto - L'identificativo alfanumerico del prodotto selezionato

quantità - Il numero di unità desiderate

Returns:

ArrayList contenente: [0] Prezzo Totale, [1] Carta di Credito, [2] Indirizzo di Spedizione [3] L'ordine da confermare.

Throws:

`AutenticazioneFallitaException` - In caso di credenziali errate.

`DisponibilitàEsauritaException` - In caso quantità richiesta eccedente quella disponibile.

`RisorsaNonTrovataException` - In caso di assenza di risorse cercate nel Database.

`DatabaseException` - In caso di errori di comunicazione con il Database. Gli errori di persistenza sono intercettate per tradurle nell' eccezione di dominio `DbException`, così da nascondere alla boundary i dettagli interni del livello di persistenza.

emettiOrdine

```
public boolean emettiOrdine(int codiceOrdine)
    throws RisorsaNonTrovataException,
           DatabaseException
```

Avvia la seconda fase del caso d'uso "Acquista Prodotto" (UCo7), in seguito alla conferma dell'ordine. La struttura dell'algoritmo è la seguente:

1. Recupero dell'ordine dal Database.
2. Pagamento
3. In caso di pagamento non andato a buon fine si procede all'annullamento dell'ordine.
4. Aggiornamento della quantità disponibile in magazzino
5. Notifica di riepilogo al cliente registrato
6. Notifica di preparazione all'impiegato
7. Modifica dello stato dell'ordine da `In_corso` a `Confermato`
8. Notifica di ritiro al fattorino
9. Modifica dello stato dell'ordine da `Confermato` a `Spedito` In un'ottica di ottimizzazione della sicurezza dei dati, il metodo accetta esclusivamente l'identificativo dell'ordine. Il sistema recupera il record dal database e applica le variazioni logiche basandosi unicamente sullo stato nativo memorizzato sul DB, azzerando i rischi di `Parameter Tampering` dal frontend.

Parameters:

`codiceOrdine` - L'identificativo intero dell'ordine confermato dal cliente.

Throws:

`RisorsaNonTrovataException` - In caso di assenza di risorse cercate nel Database.

`DatabaseException` - In caso di errori di comunicazione con il Database.

annullaOrdine

```
public boolean annullaOrdine(int codiceOrdine)
```

throws `DatabaseException`

Flusso alternativo associato al rifiuto o all'annullamento dell'acquisto da parte dell'utente. Rimuove fisicamente il record dell'ordine dalla tabella ORDINI di MySQL.

Parameters:

`codiceOrdine` - L'ID dell'ordine da eliminare in formato String

Throws:

`DatabaseException` - In caso di errori di comunicazione con il Database.

visualizzaCatalogo

```
public List <EntityProdotto> visualizzaCatalogo()  
                                throws DatabaseException
```

Recupera l'elenco completo di tutti i prodotti presenti nel catalogo, mappando il caso d'uso visualizza catalogo (UC05). Il metodo interroga il database delegando al metodo `findAll()` del Repository la restituzione di una List con tutti i prodotti presenti nello schema. Eventuali anomalie di comunicazione con il DBMS vengono intercettate e incapsulate nella classe d'eccezione di dominio `DatabaseException`.

Returns:

Una `List` contenente tutte le entità `EntityProdotto` memorizzate sul database.

Throws:

`DatabaseException` - In caso di indisponibilità del Database MySQL.

visualizzaProdotto

```
public EntityProdotto visualizzaProdotto(String id)  
                                throws DatabaseException,  
                                       RisorsaNonTrovataException
```

Recupera il singolo prodotto presente nel catalogo, mappando il caso d'uso visualizza catalogo (UC05). Verrà utilizzato per visualizzare il dettaglio di ogni prodotto all'interno del catalogo. Il metodo interroga il database delegando al metodo `findById()` del Repository la restituzione di un Optional con il prodotto presente nello schema, se trovato. Eventuali anomalie di comunicazione con il DBMS vengono intercettate e incapsulate nella classe d'eccezione di dominio `DatabaseException`. Se la risorsa non è trovata viene lanciata l'eccezione `RisorsaNonTrovataException`.

Returns:

Il prodotto memorizzato sul database.

Throws:

`RisorsaNonTrovataException` - In caso di assenza di risorse cercate nel Database.

`DatabaseException` - In caso di indisponibilità del Database MySQL.

Class BClienteRegistrato

`java.lang.Object`
`Boundary.BClienteRegistrato`

```
@CrossOrigin(origins="*")
@RestController
@RequestMapping("/api/store")
public class BClienteRegistrato
extends Object
```

Componente dello strato Boundary dell'architettura BCED, deputato all'esposizione dei servizi REST e alla gestione esclusiva del protocollo di comunicazione HTTP. Questa classe rappresenta l'unico punto di ingresso del backend che comunica direttamente con il Front-End Angular. Riceve i payload JSON transanti sulla rete, ne delega la validazione sintattica al framework tramite l'annotazione `@Valid` e mappa i dati nei Data Transfer Object preposti, garantendo l'isolamento della business logic dalle specifiche del protocollo di trasporto. La registrazione dei componenti all'interno del container IoC avviene in modalità dichiarativa sfruttando gli stereotipi standard forniti dal framework. Per poter registrare lo strato Boundary al container e dichiararla esplicitamente un Bean, utilizziamo l'annotazione:

`@RestController`: Combina `@Controller` e `@ResponseBody`, indicando al container IoC che ogni metodo restituisce i dati direttamente nel corpo della risposta HTTP serializzati in formato JSON da Jackson.

Inoltre utilizziamo le seguenti annotazioni:

`@CrossOrigin(origins = "*")`: Abilita le policy CORS (Cross-Origin Resource Sharing), consentendo le chiamate asincrone provenienti dall'applicazione Angular residente su un porto differente.

`@RequestMapping("/api/store")`: Definisce la radice uniforme degli URL per tutti gli endpoint esposti dalla Boundary.

Constructor Summary

Constructors	
Constructor	Description
BClienteRegistrato (<code>GestioneStore</code> gestioneStore)	Costruttore unico adibito alla Constructor Dependency Injection dello strato Control.

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
org.springframework.ht<?>	acquistaProdotto (@Valid RequestAcquistoDTO request org.springframework.valida	Endpoint HTTP POST deputato all'avvio della prima fase del caso d'uso "Acquista Prodotto" (UC07).
org.springframework.ht<?>	annullaOrdine (RequestAnnullamentoDTO re	Endpoint HTTP DELETE preposto all'annullamento di un acquisto in corso tra la prima e seconda fase del caso d'uso acquistaProdotto().
org.springframework.ht<?>	emettiOrdine (RequestEmissioneDTO reque	Endpoint HTTP PUT preposto alla seconda fase del caso d'uso "Acquista Prodotto" (UC07).
org.springframework.ht<?>	visualizzaCatalogo ()	Endpoint HTTP GET preposto all'esposizione dell'intero catalogo dello store.
org.springframework.ht<?>	visualizzaProdotto (String codice)	Endpoint HTTP GET preposto all'esposizione del singolo prodotto dello store.

Methods inherited from class **Object**

`clone` , `equals` , `finalize` , `getClass` , `hashCode` , `notify` , `notifyAll` , `toString` , `wait` , `wait` , `wait`

Constructor Details

BClienteRegistrato

```
public BClienteRegistrato(GestioneStore gestioneStore)
```

Costruttore unico adibito alla Constructor Dependency Injection dello strato Control.

Parameters:

`gestioneStore` - Il Bean dello strato Control da accoppiare alla Boundary

Method Details

acquistaProdotto

```

@PostMapping("/acquisto")
public org.springframework.http.ResponseEntity<?> acquistaProdotto
(@Valid @RequestBody
 @Valid RequestAcquistoDTO request,
 org.springframework.validation.BindingResult validationResult)

```

Endpoint HTTP POST deputato all'avvio della prima fase del caso d'uso "Acquista Prodotto" (UCo7). Il metodo intercetta il payload JSON inviato dal client, attiva il modulo di validazione (Jakarta Bean Validation) per intercettare gli input malformati prima che impegnino la Control. Qualora il framework rilevi una violazione dei vincoli dichiarati nel DTO, l'esecuzione non viene interrotta dall'eccezione nativa globale `MethodArgumentNotValidException`. Al contrario, l'anomalia viene registrata nel registro degli errori di `BindingResult`. Il metodo estrae programmaticamente il primo `FieldError` riscontrato e ne invoca il metodo `getDefaultMessage()` per estrarre l'esatta stringa personalizzata definita nell'attributo `message` delle annotazioni di `RequestAcquistoDTO`. Una volta che gli input sono validati, viene chiamato il metodo della Control `acquistaProdotto`. L'`ArrayList<String>` posizionale restituito viene convertito in un oggetto di risposta fortemente tipato `ResponseAcquistoDTO`.

Parameters:

`request` - Oggetto DTO contenente le credenziali, la quantità ed il prodotto validati alla Boundary

`validationResult` -

Returns:

Una `ResponseEntity` contenente il `ResponseAcquistoDTO` con stato HTTP 200 OK in caso di successo, oppure un JSON contenente il messaggio nativo dell'eccezione di dominio con stato:

HTTP 401 UNAUTHORIZED in caso di autenticazione fallita

HTTP 404 NOT_FOUND in caso di risorsa non trovata

HTTP 409 CONFLICT in caso di disponibilità esaurita

HTTP 500 INTERNAL_SERVER_ERROR in caso di errore di comunicazione con il database

emettiOrdine

```

@PutMapping("/emissione")
public org.springframework.http.ResponseEntity<?> emettiOrdine
(@RequestBody
 RequestEmissioneDTO request)

```

Endpoint HTTP PUT preposto alla seconda fase del caso d'uso "Acquista Prodotto" (UCo7). In totale aderenza con i requisiti di progetto, questo endpoint riceve il payload di ingresso mappato nel DTO `RequestEmissioneDTO`. A differenza della fase di avvio dell'acquisto, in questo metodo non viene applicata alcuna logica di validazione formale o sintattica. Tale scelta risponde a un preciso criterio di progettazione: l'identificativo dell'ordine non è un input diretto dell'utente (soggetto a errori di battitura o omissioni), bensì un dato generato nativamente dal Database

MySQL al termine della prima fase. Angular si limita a conservarlo e a rispedirlo al backend in modalità invisibile per l'utente. Il metodo estrarrà l'`int` nativo del codice ordine e invocherà il metodo `emettiOrdine` della `Control`. L'esito dell'operazione viene gestito controllando il valore booleano di ritorno o intercettando le eccezioni semantiche sollevate dal livello inferiore:

Se la `Control` restituisce `true`, il pagamento è stato approvato dal gateway simulato e lo stato dell'ordine è avanzato in modo persistente fino a 'Spedito'. Viene restituito un codice HTTP 200 OK.

Se la `Control` restituisce `false`, significa che la simulazione del pagamento ha dato esito negativo, innescando automaticamente il sotto-flusso di annullamento e rimozione del record transitorio dal DB. Viene restituito un codice HTTP 400 Bad Request.

Parameters:

`request` - Il Data Transfer Object contenente esclusivamente la chiave primaria numerica dell'ordine.

Returns:

Una `ResponseEntity` contenente una mappa JSON `Map.of("message", ...)` con il feedback testuale dell'esito dell'operazione, associata al codice di stato HTTP corrispondente.

visualizzaCatalogo

```
@GetMapping("/catalogo")
public org.springframework.http.ResponseEntity<?> visualizzaCatalogo()
```

Endpoint HTTP GET preposto all'esposizione dell'intero catalogo dello store. In aderenza alle specifiche RESTful, l'operazione di sola lettura totale viene associata al verbo HTTP GET. Lo strato Boundary riceve la collezione di entità di dominio dalla `Control`, ne esegue il mappaggio atomico all'interno di una lista di `ProdottoDTO` e restituisce il vettore, ottenendo il contratto dei dati verso il client Angular.

Returns:

Una `ResponseEntity` contenente la `List` dei prodotti configurati in formato DTO con codice di stato HTTP 200 OK; in alternativa, una mappa anonima JSON contenente la notifica dell'errore con codice HTTP 500 Internal Server Error.

visualizzaProdotto

```
@GetMapping("/catalogo/{codice}")
public org.springframework.http.ResponseEntity<?> visualizzaProdotto
(@PathVariable
 String codice)
```

Endpoint HTTP GET preposto all'esposizione del singolo prodotto dello store. In aderenza alle specifiche RESTful, l'operazione di sola lettura totale viene associata al verbo HTTP GET. Lo strato Boundary riceve l'entità di dominio dalla `Control`, e restituisce il prodotto mappato come DTO, ottenendo il contratto dei dati verso il client Angular.

Parameters:

codice -

Returns:

Un `ResponseEntity` contenente il `ProdottoDTO` con codice di stato HTTP 200 OK; in alternativa, una mappa anonima JSON contenente la notifica dell'errore con codice HTTP 500 Internal Server Error, 400 Bad Request oppure 404 NOT_FOUND.

annullaOrdine

```
@DeleteMapping("/annulla")
public org.springframework.http.ResponseEntity<?> annullaOrdine
(@RequestBody
 RequestAnnullamentoDTO request)
```

Endpoint HTTP DELETE preposto all'annullamento di un acquisto in corso tra la prima e seconda fase del caso d'uso `acquistaProdotto()`. In totale aderenza con i requisiti di progetto, questo endpoint riceve il payload di ingresso mappato nel DTO `RequestAnnullamentoDTO`.

Parameters:

request -

Returns:

Un `ResponseEntity` contenente l'esito dell'operazione di annullamento. In caso di esito positivo restituisce il codice di stato HTTP 200 OK; in alternativa, una mappa anonima JSON contenente la notifica dell'errore con codice HTTP 500 Internal Server Error.

6.7 Metriche di dimensionamento del codice

Al fine di valutare la complessità dimensionale del software, è stata condotta un'analisi quantitativa sulle linee di codice prodotte per entrambi i moduli (backend e frontend). Per evitare le approssimazioni e gli errori tipici di un conteggio manuale, la misurazione è stata effettuata tramite `cloc`, uno strumento di analisi statica open-source. Lo strumento è stato eseguito direttamente da terminale di sistema all'interno delle directory sorgenti dei due progetti. L'algoritmo di `cloc` consente di scansionare i file e di separare nettamente le righe fisiche di codice effettivo (*Lines of Code* - LOC) dalle righe lasciate vuote per motivi di formattazione e dalle righe di commento. Nello specifico, sono stati lanciati i seguenti comandi all'interno degli ambienti di sviluppo dedicati:

- **Backend:** È stato scansionato il package dei sorgenti Java, escludendo i test automatizzati e le librerie compilate, tramite l'istruzione:

```
cloc src/main/java
```

- **Frontend:** È stata analizzata la cartella dell'applicazione Angular per mappare i diversi linguaggi adoperati nella *Single Page Application* tramite l'istruzione:

```
cloc src/app
```

I dati estratti dai report del terminale sono stati inseriti all'interno della Tabella 8.

Modulo / Linguaggio	N. File	Righe Vuote	CLoC	LoC	LLoC
Backend (Java)	22	199	1.242	739	2.180
Frontend (Angular)					
– CSS	4	78	24	580	682
– TypeScript	10	57	284	289	630
– HTML	4	43	0	153	196
Totale Sistema	40	377	1.550	1.761	3.688

Tabella 8

Il numero di linee di codice previsto durante la stima dei costi, tramite la metodologia dei *Function Point Analysis* è pari a 2.385 LOC, mentre il valore effettivamente ottenuto complessivo di frontend e backend è di 1.761 LOC. In fase di analisi e conteggio dei Function Point, erano state introdotte **3 funzioni transazionali di tipo *External Output* (EO)** dedicate esplicitamente all'interfacciamento con sistemi esterni per la gestione delle notifiche e dei pagamenti. Nel passaggio alla fase di implementazione effettiva, si è ottenuta una semplificazione di tali requisiti: l'integrazione con le API di terze parti non è stata realizzata, riducendo l'interfacciamento previsto a una mera scrittura di tracciamento sui Log di console del server. Dunque la mancata elaborazione necessaria alla comunicazione con i servizi esterni giustifica la riduzione del volume di linee di codice finale rispetto alla stima predittiva iniziale.

7 Testing

7.1 Test strutturale

7.1.1 Complessità ciclomatica

acquistaProdotto()

```
public ArrayList<String> acquistaProdotto(String nomeUtente, String password,
    ↪ String codiceProdotto, int quantità) throws AutenticazioneFallitaException,
    ↪ RisorsaNonTrovataException, Disponibilit EsauritaException, DatabaseException{

    ArrayList<String> returnList = new ArrayList<>();
    returnList.add("0.0"); //Costo totale dell'ordine
    returnList.add("null"); //Carta di credito del cliente registrato
    returnList.add("null"); //Indirizzo di spedizione del cliente registrato
    returnList.add("null"); //Id dell'ordine

    try {
        //Autenticazione del Cliente Registrato (UC13):
        Optional<EntityClienteRegistrato> clienteRegistratoOpt =
            ↪ autenticazione(nomeUtente, password);
        if(clienteRegistratoOpt.isEmpty()){
            throw new AutenticazioneFallitaException("Autenticazione fallita. Nome
                ↪ utente o password errati!");
        }
        EntityClienteRegistrato clienteRegistrato = clienteRegistratoOpt.get();

        //Recupero dal Database del Prodotto
        Optional<EntityProdotto> prodottoOpt =
            ↪ prodottoRepository.findById(codiceProdotto);
        if(prodottoOpt.isEmpty()){
            throw new RisorsaNonTrovataException("Prodotto non trovato nel
                ↪ catalogo!");
        }
        EntityProdotto prodotto = prodottoOpt.get();

        //Controllo della quantit  disponibile
        if(prodotto.getQuantit ()<quantit ){
            throw new Disponibilit EsauritaException("Quantit  di prodotto non
                ↪ disponibile!");
        }

        //Calcolo del prezzo Totale
        float prezzoTotale = prodotto.getPrezzo()*quantit ;

        //Selezione del fattorino scelto per la consegna dell'ordine
        EntityFattorino fattorino = selezionaFattorino();

        //Recupero e Set all'interno dell'ArrayList del prezzoToale, Indirizzo di
        ↪ spedizione e Carta di credito
        returnList.set(0, String.valueOf(prezzoTotale));
        returnList.set(1, clienteRegistrato.getCartaDiCredito());
        returnList.set(2, clienteRegistrato.getIndirizzo());
    }
```

```

    //Costruzione dell'ordine
    EntityOrdine ordine = new EntityOrdine(LocalDate.now(), LocalTime.now(),
    ↪ prezzoTotale, quantità, prodotto, fattorino, clienteRegistrato);
    //Salvataggio dell'ordine sul Database.
    ordine = ordineRepository.save(ordine);

    //Set all'interno dell'ArrayList dell'id dell'ordine appena creato.
    returnList.set(3, String.valueOf(ordine.getId()));
}
catch (DataAccessResourceFailureException ex) {
    //Eccezione specifica in caso di connessione al Database non
    ↪ disponibile.
    throw new DatabaseException("Connessione al Database non
    ↪ disponibile!");
}
catch(DataAccessException ex){
    // Catch-all delle altre eccezioni di persistenza Spring
    throw new DatabaseException("Ops, qualcosa e' andato storto...");
}

return returnList;
}

```

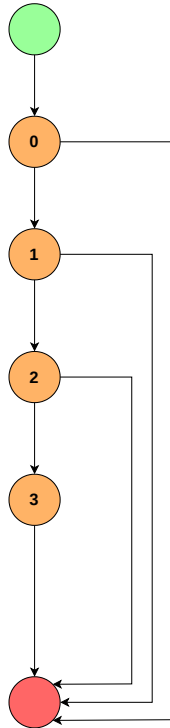
selezionaFattorino()

```

private EntityFattorino selezionaFattorino() throws RisorsaNonTrovataException{

    //Utilizziamo List in quanto restituita di default dal findAll()
    List<EntityFattorino> listaFattorini = fattorinoRepository.findAll();
    if(listaFattorini.isEmpty()){
        throw new RisorsaNonTrovataException("Nessun fattorino disponibile nel
        ↪ sistema!");
    }
    //Generazione di un indice casuale interno alla size della List
    Random random = new Random();
    int indiceCasuale = random.nextInt(listaFattorini.size());
    //Ritorno del fattorino
    return listaFattorini.get(indiceCasuale);
}

```



Numero Ciclomatico:

- numero di regioni del grafo = 4
- numero di nodi predicato (0, 1, 2) + 1 = 4
- # archi - # nodi + 2 = (7 - 5) + 2 = 4

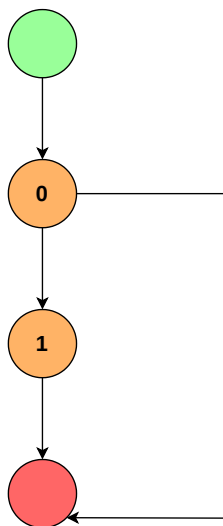
Cammini indipendenti:

1. 0
2. 0 → 1
3. 0 → 1 → 2
4. 0 → 1 → 2 → 3

Descrizione dei cammini:

- 1) **Fallimento Autenticazione:** Credenziali errate (*Nodo 0* vero). Solleva `AutenticazioneFallitaException`.
- 2) **Risorsa non trovata:** Prodotto non presente a catalogo (*Nodo 1* vero). Solleva `RisorsaNonTrovataException`.
- 3) **Quantità insufficiente:** Stock in magazzino inferiore alla richiesta (*Nodo 2* vero). Solleva `DisponibilitàEsauritaException`.
- 4) **Flusso normale:** Controlli superati (*Nodi 0, 1, 2* falsi). Calcola il prezzo, registra l'ordine (*Nodo 3*) e restituisce la lista.

Figura 8: Control Flow Graph del metodo `acquistaProdotto`.



Numero Ciclomatico:

- numero di regioni del grafo = 2
- numero di nodi predicato (0) + 1 = 2
- # archi - # nodi + 2 = (4 - 4) + 2 = 2

Cammini indipendenti:

1. 0
2. 0 → 1

Descrizione dei cammini:

- 1) **Nessun fattorino disponibile:** La lista dei fattorini recuperata dal database è vuota (*Nodo 0* vero). Solleva `RisorsaNonTrovataException`.
- 2) **Flusso normale:** Esiste almeno un fattorino registrato (*Nodo 0* falso). Genera l'indice casuale (*Nodo 1*) e restituisce l'entità.

Figura 9: Control Flow Graph del metodo `selezionaFattorino`.

7.1.2 Test di unità

Test Case ID	Metodo	Cammino CFG coperto	Stato Database	Input	Output Attesi
TU01	acquistaProdotto	• 0; • (Nodo 0 isEmpty() = True)	Database di test privo di record corrispondenti all'username inserito	{Nome Utente : "Utente", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 1}	Lancio di: AutenticazioneFallitaException. Messaggio: "Autenticazione fallita. Nome utente o password errati!"
TU02	acquistaProdotto	• $0 \rightarrow 1$; • (Nodo 0 = False, Nodo 1 isEmpty() = True)	Database di test privo di record corrispondenti al prodotto cercato	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRI", Quantità : 1}	Lancio di: RisorsaNonTrovataException. Messaggio: "Prodotto non trovato nel catalogo!"
TU03	acquistaProdotto	• $0 \rightarrow 1 \rightarrow 2$; • (Nodo 0 = False, Nodo 1 = False, Nodo 2 quantità > stock = True)	Presente il cliente "Hendrix". Presente il prodotto "KER-SHAM250-DRY" con quantità in magazzino = 10.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 11}	Lancio di: DisponibilitàEsauritaException. Messaggio: "Quantità di prodotto non disponibile!"
TU04	acquistaProdotto	• $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$; • (Nodi 0, 1, 2 = False)	Presente il cliente "Hendrix". Presente il prodotto con stock = 10. Presente il fattorino.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 1}	Ritorno di: ArrayList<String> 0 : (Prezzo totale) 1 : (Carta di Credito) 2 : (Indirizzo) 3 : (ID Ordine generato) Record inserito nella tabella ORDINI
TU05	selezionaFattorino	• 0; • (Nodo 0 sotto-grafo isEmpty() = True)	Il database di test non presenta fattorini.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 1}	Lancio di: RisorsaNonTrovataException. Messaggio: "Nessun fattorino disponibile nel sistema!"

7.2 Test Funzionale

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC01	Acquisto completato con successo con tutti gli input validi e corretti.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Prodotto esistente e disponibile nel Database. Almeno un fattorino disponibile nel Database.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Calcolo prezzo totale. Mostra indirizzo e metodo di pagamento. Mostra conferma dell'ordine.	Invio notifiche a cliente registrato, impiegato e fattorino. Ordine memorizzato e quantità di prodotto disponibile aggiornata.	Ordine emesso.	Il sistema calcola il prezzo totale e lo mostra a video con l'indirizzo ed il metodo di pagamento del cliente registrato. Le notifiche sono inviate. L'ordine è memorizzato e la quantità di prodotto aggiornata.	PASS
TC02	Inserimento di un nome utente che supera i 20 caratteri limite.	- Nome Utente > 20 caratteri [ERROR]; - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento del nome utente.	{Nome Utente : "andreavisconti1234567", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato nome utente non valido, troppo lungo!	Il sistema richiede di inserire nuovamente il nome utente.	Formato nome utente non valido, troppo lungo!	Il sistema richiede di inserire nuovamente il nome utente.	PASS

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC03	Inserimento di un nome utente contenente simboli non alfanumerici.	- Nome Utente con simboli non alfanumerici [ERROR]; - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento del nome utente.	{Nome Utente : "Hendrix_", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato nome utente non valido, contiene caratteri non consentiti!	Il sistema richiede di inserire nuovamente il nome utente.	Formato nome utente non valido, contiene caratteri non consentiti!	Il sistema richiede di inserire nuovamente il nome utente.	PASS
TC04	Inserimento di un nome utente vuoto.	- Nome Utente vuoto [ERROR]; - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento del nome utente.	{Nome Utente : "", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Inserire un Nome Utente!	Il sistema richiede di inserire nuovamente il nome utente.	Formato nome utente non valido, contiene caratteri non consentiti!	Il sistema richiede di inserire nuovamente il nome utente.	FAIL

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC05	Inserimento di una password con lunghezza non valida (numero di caratteri < 8 o > 20).	• Nome Utente (valido); • Password < 8 o > 20 caratteri [ERROR]; • Prodotto (valido); • Quantità (valido); • Conferma Ordine (valido); • DB Cliente (valido); • DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della password.	{Nome Utente : "Hendrix", Password : "N4600!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato password non valido!	Il sistema richiede di inserire nuovamente la password.	Formato password non valido!	Il sistema richiede di inserire nuovamente la password.	PASS

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC06	Inserimento di una password mancante di almeno un carattere speciale.	- Nome Utente (valido); - Password priva di almeno un carattere speciale [ERROR]; - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della password.	{Nome Utente : "Hendrix", Password : "N46007557", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Formato password non valido, carattere speciale mancante!	Il sistema richiede di inserire nuovamente la password.	Formato password non valido, carattere speciale mancante!	Il sistema richiede di inserire nuovamente la password.	PASS
TC07	Inserimento di una password vuota.	- Nome Utente (valido); - Password vuota [ERROR]; - Prodotto (valido); - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della password.	{Nome Utente : "Hendrix", Password : "", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Inserire la password!	Il sistema richiede di inserire nuovamente la password.	Formato password non valido!	Il sistema richiede di inserire nuovamente la password.	FAIL

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC08	Codice prodotto con una lunghezza diversa da 15 caratteri.	- Nome Utente (valido); - Password (valido); - Prodotto con lunghezza $\neq$ 15 caratteri [ERROR]; - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato il prodotto da acquistare.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY1", Quantità : 2, Conferma Ordine : True}	Prodotto non valido, lunghezza non valida!	Il sistema richiede di inserire nuovamente il prodotto.	Prodotto non valido, lunghezza non valida!	Il sistema richiede di inserire nuovamente il prodotto.	PASS
TC09	Codice prodotto contenente simboli non alfanumerici.	- Nome Utente (valido); - Password (valido); - Prodotto con simboli non alfanumerici [ERROR]; - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato il prodotto da acquistare.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DR?", Quantità : 2, Conferma Ordine : True}	Prodotto non valido, formattazione errata!	Il sistema richiede di inserire nuovamente il prodotto.	Prodotto non valido, formattazione errata!	Il sistema richiede di inserire nuovamente il prodotto.	PASS

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC10	Codice prodotto vuoto.	- Nome Utente (valido); - Password (valido); - Prodotto vuoto [ERROR]; - Quantità (valido); - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato il prodotto da acquistare.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "", Quantità : 2, Conferma Ordine : True}	Inserire un Prodotto!	Il sistema richiede di inserire nuovamente il prodotto.	Inserire un Prodotto!	Il sistema richiede di inserire nuovamente il prodotto.	PASS
TC11	Inserimento di una quantità richiesta pari a 0.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità = 0 [ERROR]; - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della quantità.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 0, Conferma Ordine : True}	Inserire una quantità positiva!	Il sistema richiede di inserire nuovamente la quantità.	Inserire una quantità positiva!	Il sistema richiede di inserire nuovamente la quantità.	PASS

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC12	Inserimento di una quantità richiesta minore di 0.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità < 0 [ERROR]; - Conferma Ordine (valido); - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database. Cliente Registrato esistente nel Database. Il cliente registrato ha selezionato la funzionalità di acquisto ed il sistema richiede l'inserimento della quantità.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : -1, Conferma Ordine : True}	Inserire una quantità positiva!	Il sistema richiede di inserire nuovamente la quantità.	Inserire una quantità positiva!	Il sistema richiede di inserire nuovamente la quantità.	PASS
TC13	Il cliente registrato non conferma l'ordine.	- Nome Utente (valido); - Password (valido); - Prodotto (valido); - Quantità (valido); - Conferma Ordine = false [ERROR]; - DB Cliente (valido); - DB Prodotto (valido).	Prodotto esistente nel Database con quantità disponibile sufficiente. Cliente Registrato esistente nel Database. Il sistema richiede la conferma dell'ordine.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : False}	Acquisto annullato!	L'ordine viene cancellato dal Database.	Acquisto annullato!	L'ordine viene cancellato dal Database	

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC14	Autenticazione fallita per nome utente non registrato nel Database.	• Nome Utente (valido); • Password (valido); • Prodotto (valido); • Quantità (valido); • Conferma Ordine (valido); • DB Cliente Nome Utente non presente [ERROR]; • DB Prodotto (valido).	Cliente Registrato non esistente nel Database.	{Nome Utente : "Hhendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Autenticazione fallita. Nome Utente o Password errati!	Il sistema richiede di inserire nuovamente il Nome Utente.	Autenticazione fallita. Nome Utente o Password errati!	Il sistema richiede di inserire nuovamente il Nome Utente.	PASS
TC15	Autenticazione fallita per password errata, non corrispondente al nome utente.	• Nome Utente (valido); • Password (valido); • Prodotto (valido); • Quantità (valido); • Conferma Ordine (valido); • DB Cliente Password errata [ERROR]; • DB Prodotto (valido).	Cliente Registrato non esistente nel Database.	{Nome Utente : "Hendrix", Password : "N4600755!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Autenticazione fallita. Nome Utente o Password errati!	Il sistema richiede di inserire nuovamente la password.	Autenticazione fallita. Nome Utente o Password errati!	Il sistema richiede di inserire nuovamente la password.	PASS

ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Output ottenuti	Post-condizioni Ottenute	Esito
TC16	Tentativo di acquistare più unità di quelle disponibili sul Database.	• Nome Utente (valido); • Password (valido); • Prodotto (valido); • Quantità (valido); • Conferma Ordine (valido); • DB Cliente (valido); • DB Prodotto quantità disponibile < quantità selezionata [ERROR].	Esiste una sola unità di prodotto disponibile.	{Nome Utente : "Hendrix", Password : "N46007557!", Prodotto : "KER-SHAM250-DRY", Quantità : 2, Conferma Ordine : True}	Quantità di prodotto non disponibile!	Il sistema richiede di inserire nuovamente la quantità.	Quantità di prodotto non disponibile!	Il sistema richiede di inserire nuovamente la quantità.	PASS